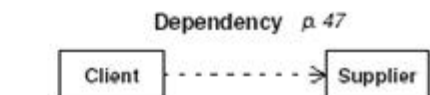
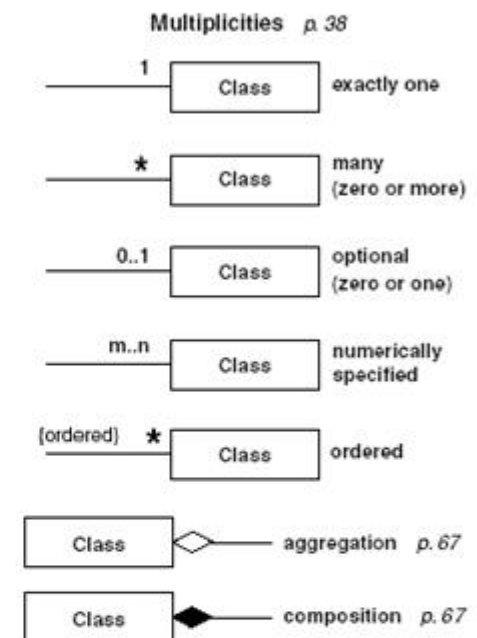
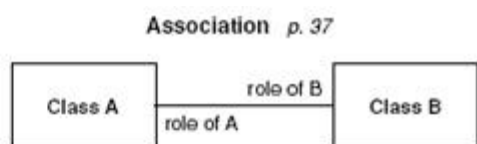
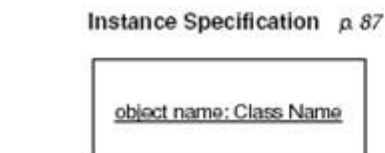
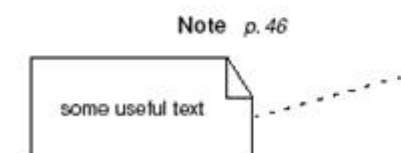
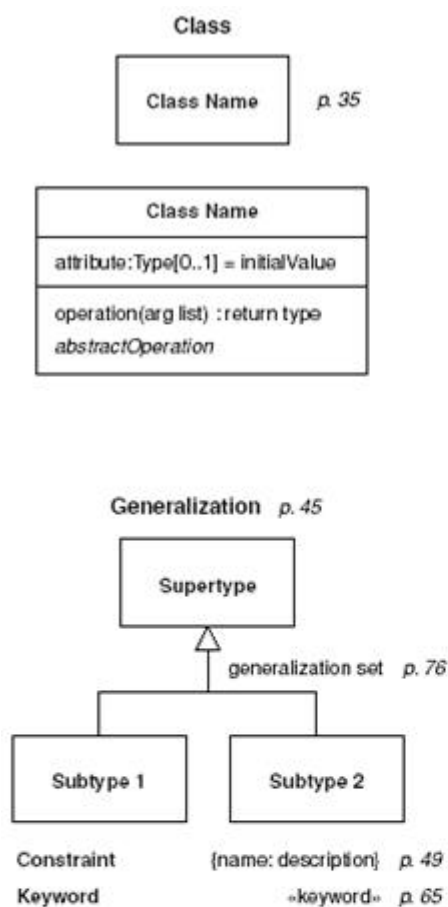
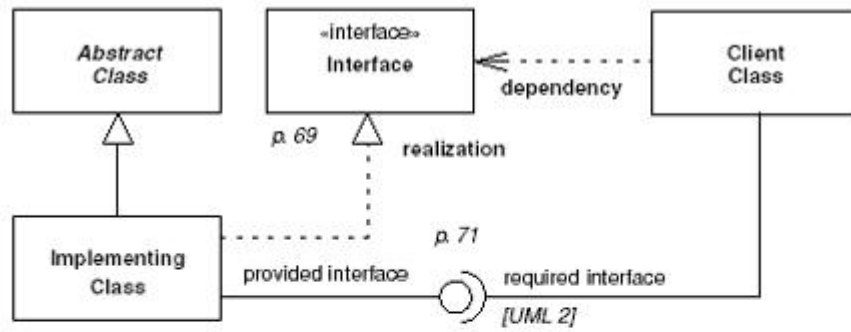


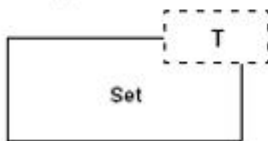
Diagrams



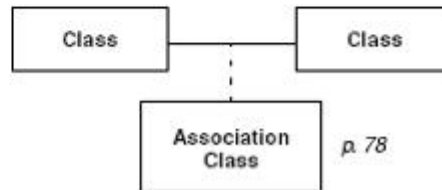
Class Diagram



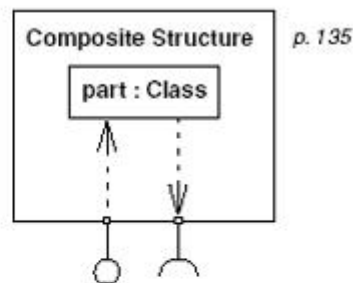
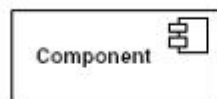
template class p. 81



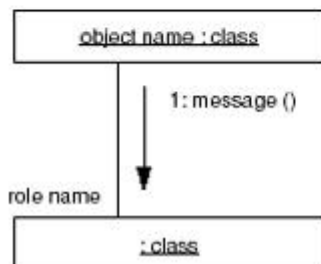
bound element



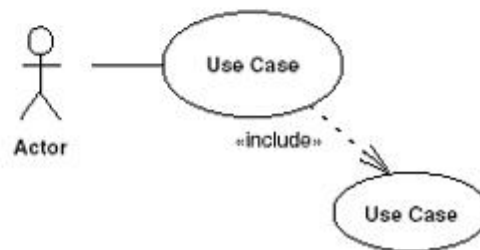
p. 139



Communication Diagram p. 131



Use Case Diagram p. 99



show you what makes a good model and how you go about constructing a model. Patterns teach by example.

When I started out, I wondered why I had to invent things from scratch. Why didn't I have handbooks to show me how to do common things? The patterns community is trying to build these handbooks.

There are now many patterns books out there, and they vary greatly in quality. My favorites are [Gang of Four], [POSA1], [POSA2], [Core J2EE Patterns], [Pont], and with suitable immodesty [Fowler, AP] and [Fowler, P of EAA]. You can also take a look at the patterns home page: <http://www.hillside.net/patterns>.

However confident you are with your process when you begin, it's essential to learn as you go along. Indeed, one of the great benefits of iterative development is that it supports frequent process improvement.

At the end of each iteration, conduct an iteration retrospective, whereby the team assembles to consider how things went and how they can be improved. A couple of hours is plenty if your iterations are short. A good way to do this is to make a list with three categories:

1. *Keep*: things that worked well that you want to ensure you continue to do
2. *Problems*: areas that aren't working well
3. *Try*: changes to your process to improve it

You can start each iteration retrospective after the first by reviewing the items from the previous session and seeing how things have changed. Don't forget the list of things to keep; it's important to keep track of things that are working. If you don't do that, you can lose a sense of perspective on the project and potentially stop paying attention to winning practices.

At the end of a project or at a major release, you may want to consider a more formal project retrospective that will last a couple of days; see <http://www.retrospectives.com/> and [Kerth] for more details. One of my biggest irritations is how organizations consistently fail to learn from their own experience and end up making expensive mistakes time and time again.

Fitting the UML into a Process

When they look at graphical modeling languages, people usually think of them in the context of a waterfall process. A waterfall process usually has documents that act as the handoffs between analysis, design, and coding phases. Graphical models can often form a major part of these documents. Indeed, many of the structured methods from the 1970s and 1980s talk a lot about analysis and design models like this.

Whether or not you use a waterfall approach, you still do the activities of analysis, design, coding, and testing. You can run an iterative project with 1-week iterations, with each week a miniwaterfall.

Using the UML doesn't necessarily imply developing documents or feeding a complex CASE tool. Many people draw UML diagrams on whiteboards only during a meeting to help communicate their ideas.

Requirements Analysis

The activity of requirements analysis involves trying to figure out what the users and customers of a software effort want the system to do. A number of UML techniques can come in handy here:

- Use cases, which describe how people interact with the system.
- A class diagram drawn from the conceptual perspective, which can be a good way of building up a rigorous vocabulary of the domain.

- An activity diagram, which can show the work flow of the organization, showing how software and human activities interact. An activity diagram can show the context for use cases and also the details of how a complicated use case works.
- A state diagram, which can be useful if a concept has an interesting life cycle, with various states and events that change that state.

When working in requirements analysis, remember that the most important thing is communication with your users and customers. Usually, they are not software people and will be unfamiliar with the UML or any other technique. Even so, I've had success using these techniques with nontechnical people. To do this, remember that it's important to keep the notation to a minimum. Don't introduce anything that specific to the software implementation.

Be prepared to break the rules of the UML at any time if it helps you communicate better. The biggest risk with using the UML in analysis is that you draw diagrams that the domain experts don't fully understand. A diagram that isn't understood by the people who know the domain is worse than useless; all it does is breed a false sense of confidence for the development team.

Design

When you are doing design, you can get more technical with your diagrams. You can use more notation and be more precise about your notation. Some useful techniques are

- Class diagrams from a software perspective. These show the classes in the software and how they interrelate.
- Sequence diagrams for common scenarios. A valuable approach is to pick the most important and interesting scenarios from the use cases and use CRC cards or sequence diagrams to figure out what happens in the software.
- Package diagrams to show the large-scale organization of the software.
- State diagrams for classes with complex life histories.
- Deployment diagrams to show the physical layout of the software.

Many of these same techniques can be used to document software once it's been written. This may help people find their way around the software if they have to work on it and are not familiar with the code.

With a waterfall life cycle, you would do these diagrams and activities as part of the phases. The end-of-phase documents usually include the appropriate UML diagrams for that activity. A waterfall style usually implies that the UML is used as a blueprint.

In an iterative style, the UML diagrams can be used in either a blueprint or a sketch style. With a blueprint, the analysis diagrams will usually be built in the iteration prior to the one that builds the functionality. Each iteration doesn't start from scratch; rather, it modifies the existing body of documents, highlighting the changes in the new iteration.

Blueprint designs are usually done early in the iteration and may be done in pieces for different bits of functionality that are targeted for the iteration. Again, iteration implies making changes to an existing model rather than building a new model each time.

Using the UML in sketch mode implies a more fluid process. One approach is to spend a couple of days at the beginning of an iteration, sketching out the design for that iteration. You can also do short design sessions at any point during the iteration, setting up a quick meeting for half an hour whenever a developer starts to tackle a nontrivial function.

With a blueprint, you expect the code implementation to follow the diagrams. A change from the blueprint is a deviation that needs review from the designers who did the blueprint. A sketch is usually treated more as a first cut at the design; if, during coding, people find that the sketch isn't exactly right, they should feel free to change the design. The implementors have to use their judgment as to whether the change needs a wider discussion to understand the full ramifications.

One of my concerns with blueprints is my own observation that it's very hard to get them right, even for a good designer. I often find that my own designs do not survive contact with coding intact. I still find UML sketches useful, but I don't find that they can be treated as absolutes.

In both modes, it makes sense to explore a number of design alternatives. It's usually best to explore alternatives in sketch mode so that you can quickly generate and change the alternatives. Once you pick a design to run with, you can either use that sketch or detail it into a blueprint.

Documentation

Once you have built the software, you can use the UML to help document what you have done. For this, I find UML diagrams useful for getting an overall understanding of a system. In doing this, however, I should stress that I do not believe in producing detailed diagrams of the whole system. To quote Ward Cunningham [Cunningham]:

Carefully selected and well-written memos can easily substitute for traditional comprehensive design documentation. The latter rarely shines except in isolated spots. Elevate those spots . . . and forget about the rest. (p. 384)

I believe that detailed documentation should be generated from the code—like, for instance, JavaDoc. You should write additional documentation to highlight important concepts. Think of these as comprising a first step for the reader before he or she goes into the code-based details. I like to structure these as prose documents, short enough to read over a cup of coffee, using UML diagrams to help illustrate the discussion. I prefer the diagrams as sketches that highlight the most important parts of the system. Obviously, the writer of the document needs to decide what is important and what isn't, but the writer is much better equipped than the reader to do that.

A package diagram makes a good logical road map of the system. This diagram helps me understand the logical pieces of the system and see the dependencies and keep them under control. A deployment diagram (see [Chapter 8](#)), which shows the high-level physical picture, may also prove useful at this stage.

Within each package, I like to see a class diagram. I don't show every operation on every class. I show only the important features that help me understand what is in there. This class diagram acts as a graphical table of contents.

The class diagram should be supported by a handful of interaction diagrams that show the most important interactions in the system. Again, selectivity is important here; remember that, in this kind of document, comprehensiveness is the enemy of comprehensibility.

If a class has complex life-cycle behavior, I draw a state machine diagram (see [Chapter 10](#)) to describe it. I do this only if the behavior is sufficiently complex, which I find doesn't happen often.

I'll often include some important code, written in a literate program style. If a particularly complex algorithm is involved, I'll consider using an activity diagram (see [Chapter 11](#)) but only if it gives me more understanding than the code alone.

If I find concepts that are coming up repeatedly, I use patterns (page 27) to capture the basic ideas.

One of the most important things to document is the design alternatives you didn't take and why you didn't do them. That's often the most forgotten but most useful piece of external documentation you can provide.

Understanding Legacy Code

The UML can help you figure out a gnarly bunch of unfamiliar code in a couple of ways. Building a sketch of key facts can act as a graphical note-taking mechanism that helps you capture important information as you learn about it. Sketches of key classes in a package and their key interactions can help clarify what's going on.

With modern tools, you can generate detailed diagrams for key parts of a system. Don't use these tools to generate big paper reports; instead, use them to drill into key areas as you are exploring the code itself. A particularly nice capability is that of generating a sequence diagram to see how multiple objects collaborate in handling a complex method.

Choosing a Development Process

I'm strongly in favor of iterative development processes. As I've said in this book before: You should use iterative development only on projects that you want to succeed.

Perhaps that's a bit glib, but as I get older, I get more aggressive about using iterative development. Done well, it is an essential technique, one you can use to expose risk early and to obtain better control over development. It is not the same as having no management, although to be fair, I should point out that some have used it that way. It does need to be well planned. But it is a solid approach, and every OO development book encourages using it—for good reason.

You should not be surprised to hear that as one the authors of the Manifesto for Agile Software Development, I'm very much a fan of agile approaches. I've also had a lot of positive experiences with Extreme Programming, and certainly you should consider its practices very seriously.

Where to Find Out More

Books on software process have always been common, and the rise of agile software development has led to many new books. Overall, my favorite book on process in general is [McConnell]. He gives a broad and practical coverage of many of the issues involved in software development and a long list of useful practices.

From the agile community, [Cockburn, agile] and [Highsmith] provide a good overview. For a lot of good advice about applying the UML in an agile way, see [Ambler].

One of the most popular agile methods is Extreme Programming (XP), which you can delve into via such Web sites as <http://xprogramming.com> and <http://www.extremeprogramming.org>. XP has spawned many books, which is why I now refer to it as the formerly lightweight methodology. The usual starting point is [Beck].

Although it's written for XP, [Beck and Fowler] gives more details on planning an iterative project. Much of this is also covered by the other XP books, but if you're interested only in the planning aspect, this would be a good choice.

For more information on the Rational Unified Process, my favorite introduction is [Kruchten].

Chapter 3. Class Diagrams: The Essentials

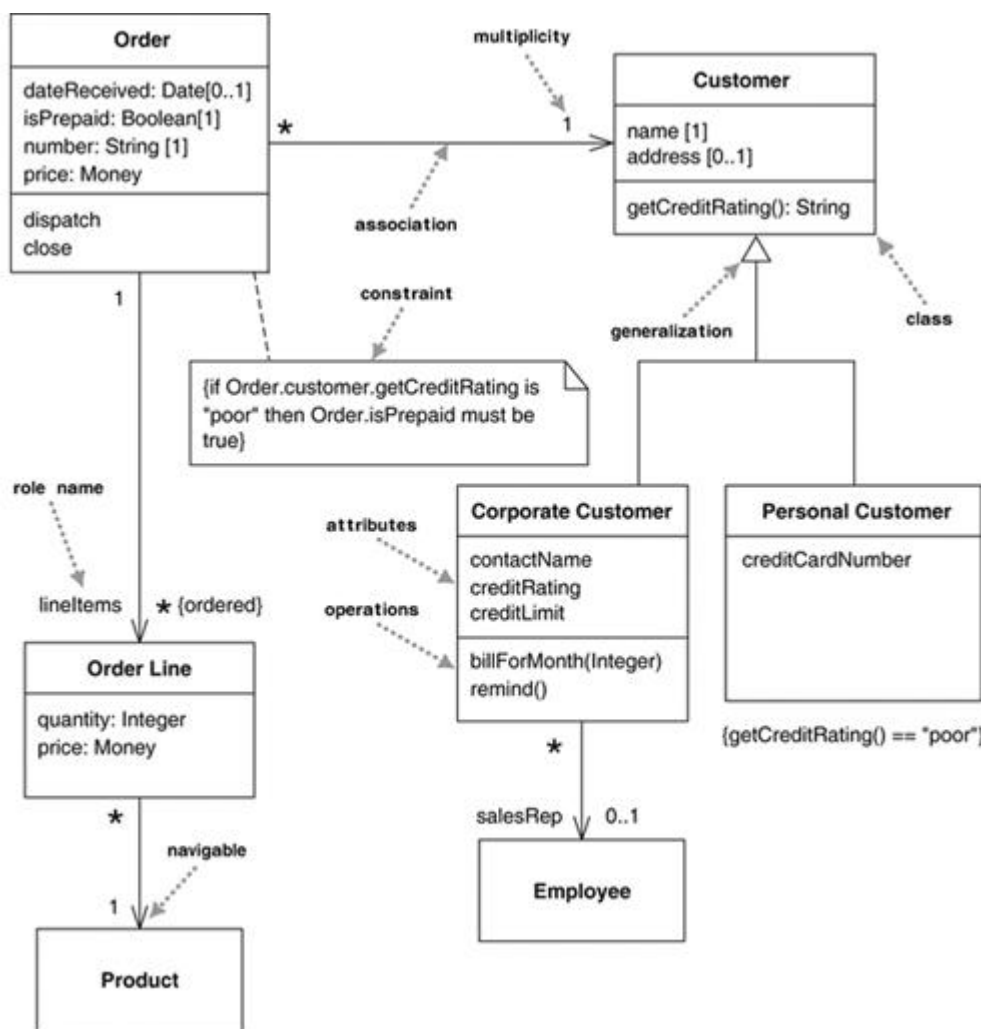
If someone were to come up to you in a dark alley and say, "Psst, wanna see a UML diagram?" that diagram would probably be a class diagram. The majority of UML diagrams I see are class diagrams.

The class diagram is not only widely used but also subject to the greatest range of modeling concepts. Although the basic elements are needed by everyone, the advanced concepts are used less often. Therefore, I've broken my discussion of class diagrams into two parts: the essentials (this chapter) and the advanced ([Chapter 5](#)).

A class diagram describes the types of objects in the system and the various kinds of static relationships that exist among them. Class diagrams also show the properties and operations of a class and the constraints that apply to the way objects are connected. The UML uses the term feature as a general term that covers properties and operations of a class.

[Figure 3.1](#) shows a simple class model that would not surprise anyone who has worked with order processing. The boxes in the diagram are classes, which are divided into three compartments: the name of the class (in bold), its attributes, and its operations. [Figure 3.1](#) also shows two kinds of relationships between classes: associations and generalizations.

Figure 3.1. A simple class diagram



Properties

Properties represent structural features of a class. As a first approximation, you can think of properties as corresponding to fields in a class. The reality is rather involved, as we shall see, but that's a reasonable place to start.

Properties are a single concept, but they appear in two quite distinct notations: attributes and associations. Although they look quite different on a diagram, they are really the same thing.

Attributes

The attribute notation describes a property as a line of text within the class box itself. The full form of an attribute is:

visibility name: type multiplicity = default {property-string}

An example of this is:

```
- name: String [1] = "Untitled" {readOnly}
```

Only the **name** is necessary.

- This **visibility** marker indicates whether the attribute is public (+) or private (-); I'll discuss other visibilities on page 83.
- The **name** of the attribute—how the class refers to the attribute—roughly corresponds to the name of a field in a programming language.
- The **type** of the attribute indicates a restriction on what kind of object may be placed in the attribute. You can think of this as the type of a field in a programming language.
- I'll explain **multiplicity** on page 38.
- The **default value** is the value for a newly created object if the attribute isn't specified during creation.
- The **{property-string}** allows you to indicate additional properties for the attribute. In the example, I used **{readOnly}** to indicate that clients may not modify the property. If this is missing, you can usually assume that the attribute is modifiable. I'll describe other property strings as we go.

Associations

The other way to notate a property is as an association. Much of the same information that you can show on an attribute appears on an association. [Figures 3.2](#) and [3.3](#) show the same properties represented in the two different notations.

Figure 3.2. Showing properties of an order as attributes

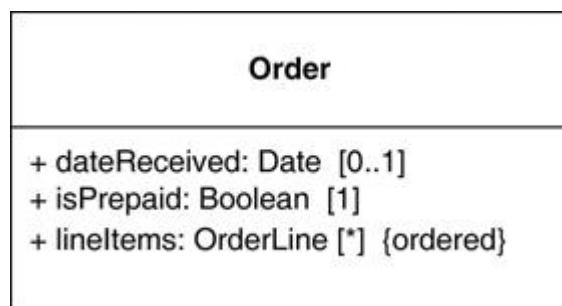
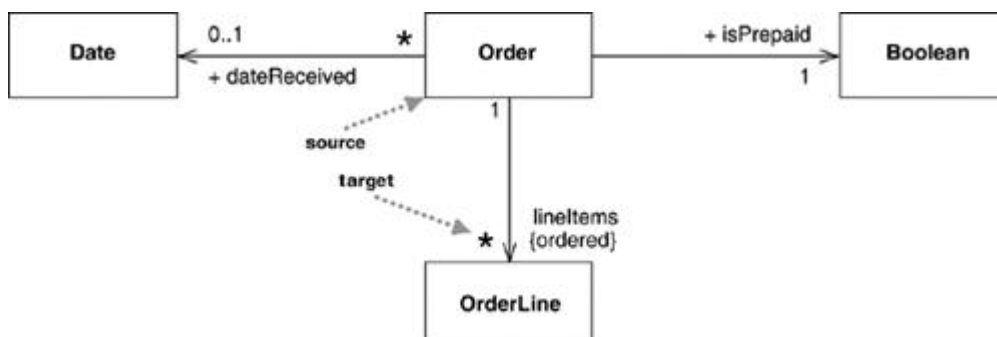


Figure 3.3. Showing properties of an order as associations



An association is a solid line between two classes, directed from the source class to the target class. The name of the property goes at the target end of the association, together with its multiplicity. The target end of the association links to the class that is the type of the property.

Although most of the same information appears in both notations, some items are different. In particular, associations can show multiplicities at both ends of the line.

With two notations for the same thing, the obvious question is, Why should you use one or the other? In general, I tend to use attributes for small things, such as dates or Booleans—in general, value types (page 73)—and associations for more significant classes, such as customers and orders. I also tend to prefer to use class boxes for classes that are significant for the diagram, which leads to using associations, and attributes for things less important for that diagram. The choice is much more about emphasis than about any underlying meaning.

When to Use Class Diagrams

Class diagrams are the backbone of the UML, so you will find yourself using them all the time. This chapter covers the basic concepts; [Chapter 5](#) discusses many of the advanced concepts.

The trouble with class diagrams is that they are so rich, they can be overwhelming to use. Here are a few tips.

- Don't try to use all the notations available to you. Start with the simple stuff in this chapter: classes, associations, attributes, generalization, and constraints. Introduce other notations from [Chapter 5](#) only when you need them.
- I've found conceptual class diagrams very useful in exploring the language of a business. For this to work, you have to work hard on keeping software out of the discussion and keeping the notation very simple.
- Don't draw models for everything; instead, concentrate on the key areas. It is better to have a few diagrams that you use and keep up to date than to have many forgotten, obsolete models.

The biggest danger with class diagrams is that you can focus exclusively on structure and ignore behavior. Therefore, when drawing class diagrams to understand software, always do them in conjunction with some form of behavioral technique. If you're going well, you'll find yourself swapping between the techniques frequently.

Where to Find Out More

All the general UML books I mentioned in [Chapter 1](#) talk about class diagrams in more detail. Dependency management is a critical feature of larger projects. The best book on this topic is [Martin].

Multiplicity

The multiplicity of a property is an indication of how many objects may fill the property. The most common multiplicities you will see are

- 1 (An order must have exactly one customer.)
- 0..1 (A corporate customer may or may not have a single sales rep.)
- * (A customer need not place an Order and there is no upper limit to the number of Orders a Customer may place—zero or more orders.)

More generally, multiplicities are defined with a lower bound and an upper bound, such as 2..4 for players of a game of canasta. The lower bound may be any positive number or zero; the upper is

any positive number or * (for unlimited). If the lower and upper bounds are the same, you can use one number; hence, 1 is equivalent to 1..1. Because it's a common case, * is short for 0..*.

In attributes, you come across various terms that refer to the multiplicity.

- Optional implies a lower bound of 0.
- Mandatory implies a lower bound of 1 or possibly more.
- Single-valued implies an upper bound of 1.
- Multivalued implies an upper bound of more than 1: usually *.

If I have a multivalued property, I prefer to use a plural form for its name.

By default, the elements in a multivalued multiplicity form a set, so if you ask a customer for its orders, they do not come back in any order. If the ordering of the orders in association has meaning, you need to add `{ordered}` to the association end. If you want to allow duplicates, add `{nonunique}`. (If you want to explicitly show the default, you can use `{unordered}` and `{unique}`.) You may also see collection-oriented names, such as `{bag}` for unordered, nonunique.

UML 1 allowed discontinuous multiplicities, such as 2, 4 (meaning 2 or 4, as in cars in the days before minivans). Discontinuous multiplicities weren't very common and UML 2 removed them.

The default multiplicity of an attribute is [1]. Although this is true in the meta-model, you can't assume that an attribute in a diagram that's missing a multiplicity has a value of [1], as the diagram may be suppressing the multiplicity information. As a result, I prefer to explicitly state a [1] multiplicity if it's important.

Programming Interpretation of Properties

As with anything else in the UML, there's no one way to interpret properties in code. The most common software representation is that of a field or property of your programming language. So the Order Line class from [Figure 3.1](#) would correspond to something like the following in Java:

```
public class OrderLine...
    private int quantity;
    private Money price;
    private Order order;
    private Product product
```

In a language like C#, which has properties, it would correspond to:

```
public class OrderLine ...
    public int Quantity;
    public Money Price;
    public Order Order;
    public Product Product;
```

Note that an attribute typically corresponds to public properties in a language that supports properties but to private fields in a language that does not. In a language without properties, you may see the fields exposed through accessor (getting and setting) methods. A read-only attribute will have no setting method (with fields) or set action (for properties). Note that if you don't give a name for a property, it's common to use the name of the target class.

Using private fields is a very implementation-focused interpretation of the diagram. A more interface-oriented interpretation might instead concentrate on the getting methods rather than the

underlying data. In this case, we might see the Order Line's attributes corresponding to the following methods:

```
public class OrderLine...
    private int quantity;
    private Product product;
    public int getQuantity() {
        return quantity;
    }
    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
    public Money getPrice() {
        return product.getPrice().multiply(quantity);
    }
}
```

In this case, there is no data field for price; instead, it's a computed value. But as far as clients of the Order Line class are concerned, it looks the same as a field. Clients can't tell what is a field and what is computed. This information hiding is the essence of encapsulation.

If an attribute is multivalued, this implies that the data concerned is a collection. So an Order class would refer to a collection of Order Lines. Because this multiplicity is ordered, that collection must be ordered, (such as a List in Java or an IList in .NET). If the collection is unordered, it should, strictly, have no meaningful order and thus be implemented with a set, but most people implement unordered attributes as lists as well. Some people use arrays, but the UML implies an unlimited upper bound, so I almost always use a collection for data structure.

Multivalued properties yield a different kind of interface to single-valued properties (in Java):

```
class Order {
    private Set lineItems = new HashSet();
    public Set getLineItems() {
        return Collections.unmodifiableSet(lineItems);
    }
    public void addLineItem (OrderItem arg) {
        lineItems.add (arg);
    }
    public void removeLineItem (OrderItem arg) {
        lineItems.remove(arg);
    }
}
```

In most cases, you don't assign to a multivalued property; instead, you update with add and remove methods. In order to control its Line Items property, the order must control membership of that collection; as a result, it shouldn't pass out the naked collection. In this case, I used a protection proxy to provide a read-only wrapper to the collection. You can also provide a nonupdatable iterator or make a copy. It's okay for clients to modify the member objects, but the clients shouldn't directly change the collection itself.

Because multivalued attributes imply collections, you almost never see collection classes on a class diagram. You would show them only in very low level implementation diagrams of collections themselves.

You should be very afraid of classes that are nothing but a collection of fields and their accessors. Object-oriented design is about providing objects that are able to do rich behavior, so they shouldn't be simply providing data to other objects. If you are making repeated calls for data by using accessors, that's a sign that some behavior should be moved to the object that has the data.

These examples also reinforce the fact that there is no hard-and-fast correspondence between the UML and code, yet there is a similarity. Within a project team, team conventions will lead to a closer correspondence.

Whether a property is implemented as a field or as a calculated value, it represents something an object can always provide. You shouldn't use a property to model a transient relationship, such as an object that is passed as a parameter during a method call and used only within the confines of that interaction.

Bidirectional Associations

The associations we've looked at so far are called unidirectional associations. Another common kind of association is a bidirectional association, such as [Figure 3.4](#).

Figure 3.4. A bidirectional association



A bidirectional association is a pair of properties that are linked together as inverses. The Car class has property `owner:Person[1]`, and the Person class has a property `cars:Car[*]`. (Note how I named the `cars` property in the plural form of the property's type, a common but non-normative convention.)

The inverse link between them implies that if you follow both properties, you should get back to a set that contains your starting point. For example, if I begin with a particular MG Midget, find its owner, and then look at its owner's cars, that set should contain the Midget that I started from.

As an alternative to labeling an association by a property, many people, particularly if they have a data-modeling background, like to label an association by using a verb phrase ([Figure 3.5](#)) so that the relationship can be used in a sentence. This is legal and you can add an arrow to the association to avoid ambiguity. Most object modelers prefer to use a property name, as that corresponds better to responsibilities and operations.

Figure 3.5. Using a verb phrase to name an association



Some people name every association in some way. I choose to name an association only when doing so improves understanding. I've seen too many associations with such names as "has" or "is related to."

In [Figure 3.4](#), the bidirectional nature of the association is made obvious by the navigability arrows at both ends of the association. [Figure 3.5](#) has no arrows; the UML allows you to use this form either to indicate a bidirectional association or when you aren't showing navigability. My preference is to use the double-headed arrow of [Figure 3.4](#) when you want to make it clear that you have a bidirectional association.

Implementing a bidirectional association in a programming language is often a little tricky because you have to be sure that both properties are kept synchronized. Using C#, I use code along these lines to implement a bidirectional association:

```
class Car...
```

```

    public Person Owner {
        get {return _owner;}
        set {
            if (_owner != null) _owner.friendCars().Remove(this);
            _owner = value;
            if (_owner != null) _owner.friendCars().Add(this);
        }
    }
    private Person _owner;
    ...
class Person ...
    public IList Cars {
        get {return ArrayList.ReadOnly(_cars);}
    }
    public void AddCar(Car arg) {
        arg.Owner = this;
    }
    private IList _cars = new ArrayList();
    internal IList friendCars() {
        //should only be used by Car.Owner
        return _cars;
    }
    ....

```

The primary thing is to let one side of the association—a single-valued side, if possible—control the relationship. For this to work, the slave end (Person) needs to leak the encapsulation of its data to the master end. This adds to the slave class an awkward method, which shouldn't really be there, unless the language has fine-grained access control. I've used the naming convention of "friend" here as a nod to C++, where the master's setter would indeed be a friend. Like much property code, this is pretty boilerplate stuff, which is why many people prefer to use some form of code generation to produce it.

In conceptual models, navigability isn't an important issue, so I don't show any navigability arrows on conceptual models.

Operations

Operations are the actions that a class knows to carry out. Operations most obviously correspond to the methods on a class. Normally, you don't show those operations that simply manipulate properties, because they can usually be inferred.

The full UML syntax for operations is:

```
visibility name (parameter-list) : return-type {property-string}
```

- This **visibility** marker is public (+) or private (-); others on page 83.
- The **name** is a string.
- The **parameter-list** is the list of parameters for the operation.
- The **return-type** is the type of the returned value, if there is one.
- The **property-string** indicates property values that apply to the given operation.

The parameters in the parameter list are notated in a similar way to attributes. The form is:

```
direction name: type = default value
```

- The **name**, **type**, and **default value** are the same as for attributes.
- The **direction** indicates whether the parameter is input (**in**), output (**out**) or both (**inout**). If no direction is shown, it's assumed to be **in**.

An example operation on account might be:

```
+ balanceOn (date: Date) : Money
```

With conceptual models, you shouldn't use operations to specify the interface of a class. Instead, use them to indicate the principal responsibilities of that class, perhaps using a couple of words summarizing a CRC responsibility (page 65).

I often find it useful to distinguish between operations that change the state of the system and those that don't. UML defines a query as an operation that gets a value from a class without changing the system state—in other words, without side effects. You can mark such an operation with the property string `{query}`. I refer to operations that do change state as modifiers, also called commands.

Strictly, the difference between query and modifiers is whether they change the observable state [Meyer]. The observable state is what can be perceived from the outside. An operation that updates a cache would alter the internal state but would have no effect that's observable from the outside.

I find it helpful to highlight queries, as you can change the order of execution of queries and not change the system behavior. A common convention is to try to write operations so that modifiers do not return a value; that way, you can rely on the fact that operations that return a value are queries. [Meyer] refers to this as the Command-Query separation principle. It's sometimes awkward to do this all the time, but you should do it as much as you can.

Other terms you sometimes see are getting methods and setting methods. A getting method returns a value from a field (and does nothing else). A setting method puts a value into a field (and does nothing else). From the outside, a client should not be able to tell whether a query is a getting method or a modifier is a setting method. Knowledge of getting and setting methods is entirely internal to the class.

Another distinction is between operation and method. An operation is something that is invoked on an object—the procedure declaration—whereas a method is the body of a procedure. The two are different when you have polymorphism. If you have a supertype with three subtypes, each of which overrides the supertype's `getPrice` operation, you have one operation and four methods that implement it.

People usually use the terms *operation* and *method* interchangeably, but there are times when it is useful to be precise about the difference.

Generalization

A typical example of generalization involves the personal and corporate customers of a business. They have differences but also many similarities. The similarities can be placed in a general Customer class (the supertype), with Personal Customer and Corporate Customer as subtypes.

This phenomenon is also subject to various interpretations at the various perspectives of modeling. Conceptually, we can say that Corporate Customer is a subtype of Customer if all instances of Corporate Customer are also, by definition, instances of Customer. A Corporate Customer is then a special kind of Customer. The key idea is that everything we say about a Customer—associations, attributes, operations—is true also for a Corporate Customer.

With a software perspective, the obvious interpretation is inheritance: The Corporate Customer is a subclass of Customer. In mainstream OO languages, the subclass inherits all the features of the superclass and may override any superclass methods.

An important principle of using inheritance effectively is substitutability. I should be able to substitute a Corporate Customer within any code that requires a Customer, and everything should work fine. Essentially, this means that if I write code assuming I have a Customer, I can freely use any subtype of Customer. The Corporate Customer may respond to certain commands differently

from another Customer, using polymorphism, but the caller should not need to worry about the difference. (For more on this, see the Liskov Substitution Principle (LSP) in [Martin].)

Although inheritance is a powerful mechanism, it brings in a lot of baggage that isn't always needed to achieve substitutability. A good example of this was in the early days of Java, when many people didn't like the implementation of the built-in Vector class and wanted to replace it with something lighter. However, the only way they could produce a class that was substitutable for Vector was to subclass it, and that meant inheriting a lot of unwanted data and behavior.

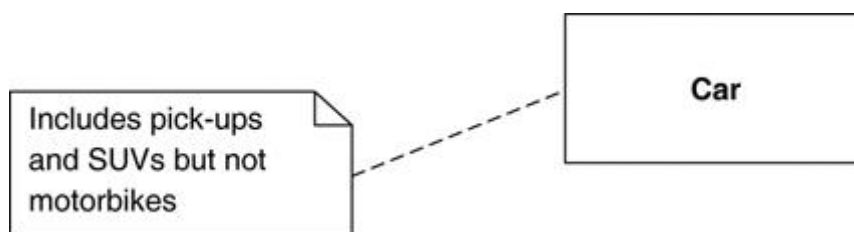
Many other mechanisms can be used to provide substitutable classes. As a result, many people like to differentiate between subtyping, or interface inheritance, and subclassing, or implementation inheritance. A class is a subtype if it is substitutable for its supertype, whether or not it uses inheritance. Subclassing is used as a synonym for regular inheritance.

Many other mechanisms are available that allow you to have subtyping without subclassing. Examples are implementing an interface (page 69) and many of the standard design patterns [Gang of Four].

Notes and Comments

Notes are comments in the diagrams. Notes can stand on their own, or they can be linked with a dashed line to the elements they are commenting ([Figure 3.6](#)). They can appear in any kind of diagram.

Figure 3.6. A note is used as a comment on one or more diagram elements



The dashed line can sometimes be awkward because you can't position exactly where this line ends. So a common convention is to put a very small open circle at the end of the line. Sometimes, it's useful to have an in-line comment on a diagram element. You can do this by prefixing the text with two dashes: --.

Dependency

A dependency exists between two elements if changes to the definition of one element (the supplier) may cause changes to the other (the client). With classes, dependencies exist for various reasons: One class sends a message to another; one class has another as part of its data; one class mentions another as a parameter to an operation. If a class changes its interface, any message sent to that class may no longer be valid.

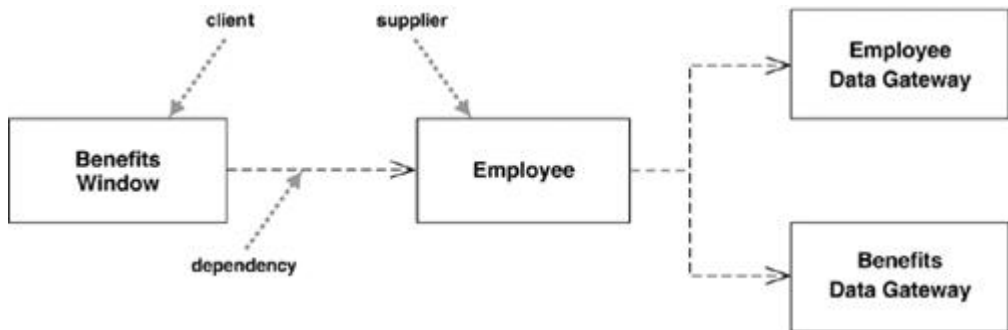
As computer systems grow, you have to worry more and more about controlling dependencies. If dependencies get out of control, each change to a system has a wide ripple effect as more and more things have to change. The bigger the ripple, the harder it is to change anything.

The UML allows you to depict dependencies between all sorts of elements. You use dependencies whenever you want to show how changes in one element might alter other elements.

[Figure 3.7](#) shows some dependencies that you might find in a multilayered application. The Benefits Window class—a user interface, or presentation class—is dependent on the Employee class: a

domain object that captures the essential behavior of the system—in this case, business rules. This means that if the employee class changes its interface, the Benefits Window may have to change.

Figure 3.7. Example dependencies



The important thing here is that the dependency is in only one direction and goes from the presentation class to the domain class. This way, we know that we can freely alter the Benefits Window without those changes having any effect on the Employee or other domain objects. I've found that a strict separation of presentation and domain logic, with the presentation depending on the domain but not vice versa, has been a valuable rule for me to follow.

A second notable thing from this diagram is that there is no direct dependency from the Benefits Window to the two Data Gateway classes. If these classes change, the Employee class may have to change. But if the change is only to the implementation of the Employee class, not its interface, the change stops there.

The UML has many varieties of dependency, each with particular semantics and keywords. The basic dependency that I've outlined here is the one I find the most useful, and I usually use it without keywords. To add more detail, you can add an appropriate keyword ([Table 3.1](#)).

The basic dependency is not a transitive relationship. An example of a transitive relationship is the "larger beard" relationship. If Jim has a larger beard than Grady, and Grady has a larger beard than Ivar, we can deduce that Jim has a larger beard than Ivar. Some kind of dependencies, such as substitute, are transitive, but in most cases there is a significant difference between direct and indirect dependencies, as there is in [Figure 3.7](#).

Many UML relationships imply a dependency. The navigable association from Order to Customer in [Figure 3.1](#) means that Order is dependent on Customer. A subclass is dependent on its superclass but not vice versa.

Table 3.1. Selected Dependency Keywords

Keyword	Meaning
«call»	The source calls an operation in the target.
«create»	The source creates instances of the target.
«derive»	The source is derived from the target.
«instantiate»	The source is an instance of the target. (Note that if the source is a class, the class itself is an instance of the class class; that is, the target class is a metaclass).
«permit»	The target allows the source to access the target's private features.
«realize»	The source is an implementation of a specification or interface defined by the target (page 69).
«refine»	Refinement indicates a relationship between different semantic levels; for example, the source might be a design class and the target the corresponding analysis class.
«substitute»	The source is substitutable for the target (page 45).
«trace»	Used to track such things as requirements to classes or how changes in one model

Table 3.1. Selected Dependency Keywords

Keyword	Meaning
	link to changes elsewhere.
«use»	The source requires the target for its implementation.

Your general rule should be to minimize dependencies, particularly when they cross large areas of a system. In particular, you should be wary of cycles, as they can lead to a cycle of changes. I'm not super strict on this. I don't mind mutual dependencies between closely related classes, but I do try to eliminate cycles at a broader level, particularly between packages.

Trying to show all the dependencies in a class diagram is an exercise in futility; there are too many and they change too much. Be selective and show dependencies only when they are directly relevant to the particular topic that you want to communicate. To understand and control dependencies, you are best off using them with package diagrams (pages 89).

The most common case I use for dependencies with classes is when illustrating a transient relationship, such as when one object is passed to another as a parameter. You may see these used with keywords «parameter», «local», and «global». You may also see these keywords on associations in UML 1 models, in which case they indicate transient links, not properties. These keywords are not part of UML 2.

Dependencies can be determined by looking at code, so tools are ideal for doing dependency analysis. Getting a tool to reverse engineer pictures of dependencies is the most useful way to use this bit of the UML.

Constraint Rules

Much of what you are doing in drawing a class diagram is indicating constraints. [Figure 3.1](#) indicates that an Order can be placed only by a single Customer. The diagram also implies that each Line Item is thought of separately: You say "40 brown widgets, 40 blue widgets, and 40 red widgets," not "120 things" on the Order. Further, the diagram says that Corporate Customers have credit limits but Personal Customers do not.

The basic constructs of association, attribute, and generalization do much to specify important constraints, but they cannot indicate every constraint. These constraints still need to be captured; the class diagram is a good place to do that.

The UML allows you to use anything to describe constraints. The only rule is that you put them inside braces ({}). You can use natural language, a programming language, or the UML's formal Object Constraint Language (OCL) [Warmer and Kleppe], which is based on predicate calculus. Using a formal notation avoids the risk of misinterpretation due to an ambiguous natural language. However, it introduces the risk of misinterpretation due to writers and readers not really understanding OCL. So unless you have readers who are comfortable with predicate calculus, I'd suggest using natural language.

Optionally, you can name a constraint by putting the name first, followed by a colon; for example, {disallow incest: husband and wife must not be siblings}.

Design by Contract

Design by Contract is a design technique developed by Bertrand Meyer [Meyer]. The technique is a central feature of the Eiffel language he developed. Design by Contract is not specific to Eiffel, however; it is a valuable technique that can be used with any programming language.

At the heart of Design by Contract is the assertion. An assertion is a Boolean statement

stated by rolling them up into higher-level statements.

Many people stress the importance of role playing, whereby each person on the team plays the role of one or more classes. I've never seen Ward Cunningham do that, and I find that role playing gets in the way.

Books have been written on CRC, but I've found that they never really get to the heart of the technique. The original paper on CRC, written with Kent Beck, is [Beck and Cunningham]. To learn more about both CRC cards and responsibilities in design, take a look at [Wirfs-Brock].

Chapter 5. Class Diagrams: Advanced Concepts

The concepts described in [Chapter 3](#) correspond to the key notations in class diagrams. Those concepts are the first ones to understand and become familiar with, as they will comprise 90 percent of your effort in building class diagrams.

The class diagram technique, however, has bred dozens of notations for additional concepts. I find that I don't use these all the time, but they are handy when they are appropriate. I'll discuss them one at a time and point out some of the issues in using them.

You'll probably find this chapter somewhat heavy going. The good news is that during your first pass through the book, you can safely skip this chapter and come back to it later.

Keywords

One of the challenges of a graphical language is that you have to remember what the symbols mean. With too many, users find it very difficult to remember what all the symbols mean. So the UML often tries to reduce the number of symbols and use keywords instead. If you find that you need a modeling construct that isn't in the UML but is similar to something that is, use the symbol of the existing UML construct but mark it with a keyword to show that you have something different

An example of this is the interface. A UML interface (page 69) is a class that has only public operations, with no method bodies. This corresponds to interfaces in Java, COM (Component Object Module), and CORBA. Because it's a special kind of class, it is shown using the class icon with the keyword `«interface»`. Keywords are usually shown as text between guillemets. As an alternative to keywords, you can use special icons, but then you run into the issue of everyone having to remember what they mean.

Some keywords, such as `{abstract}`, show up in curly brackets. It's never really clear what should technically be in guillemets and what should be in curlies. Fortunately, if you get it wrong, only serious UML weenies will notice—or care.

Some keywords are so common that they often get abbreviated: `«interface»` often gets abbreviated to `«I»` and `{abstract}` to `{A}`. Such abbreviations are very useful, particularly on whiteboards, but nonstandard, so if you use them, make sure you find a spot to spell out what they mean.

In UML 1, the guillemets were used mainly for stereotypes. In UML 2, stereotypes are defined very tightly, and describing what is and isn't a stereotype is beyond the scope of this book. However,

because of UML 1, many people use the term *stereotype* to mean the same as *keyword*, although that is no longer correct.

Stereotypes are used as part of profiles. A profile takes a part of the UML and extends it with a coherent group of stereotypes for a particular purpose, such as business modeling. The full semantics of profiles are beyond this book. Unless you are into serious meta-model design, you're unlikely to need to create one yourself. You're more likely to use one created for a specific modeling purpose, but fortunately, use of a profile doesn't require you to know the gory details of how they are tied into the meta-model.

Classification and Generalization

I often hear people talk about subtyping as the *is a* relationship. I urge you to beware of that way of thinking. The problem is that the phrase *is a* can mean different things.

Consider the following phrases.

1. Shep is a Border Collie.
2. A Border Collie is a Dog.
3. Dogs are Animals.
4. A Border Collie is a Breed.
5. Dog is a Species.

Now try combining the phrases. If I combine phrases 1 and 2, I get "Shep is a Dog"; 2 and 3 taken together yield "Border Collies are Animals." And 1 plus 2 plus 3 gives me "Shep is an Animal." So far, so good. Now try 1 and 4: "Shep is a Breed." The combination of 2 and 5 is "A Border Collie is a Species." These are not so good.

Why can I combine some of these phrases and not others? The reason is that some are classification—the object Shep is an instance of the type Border Collie—and some are generalization—the type Border Collie is a subtype of the type Dog. Generalization is transitive; classification is not. I can combine a classification followed by a generalization but not vice versa.

I make this point to get you to be wary of *is a*. Using it can lead to inappropriate use of subclassing and confused responsibilities. Better tests for subtyping in this case would be the phrases "Dogs are kinds of Animals" and "Every instance of a Border Collie is an instance of a Dog."

The UML uses the generalization symbol to show generalization. If you need to show classification, use a dependency with the `<<instantiate>>` keyword.

Multiple and Dynamic Classification

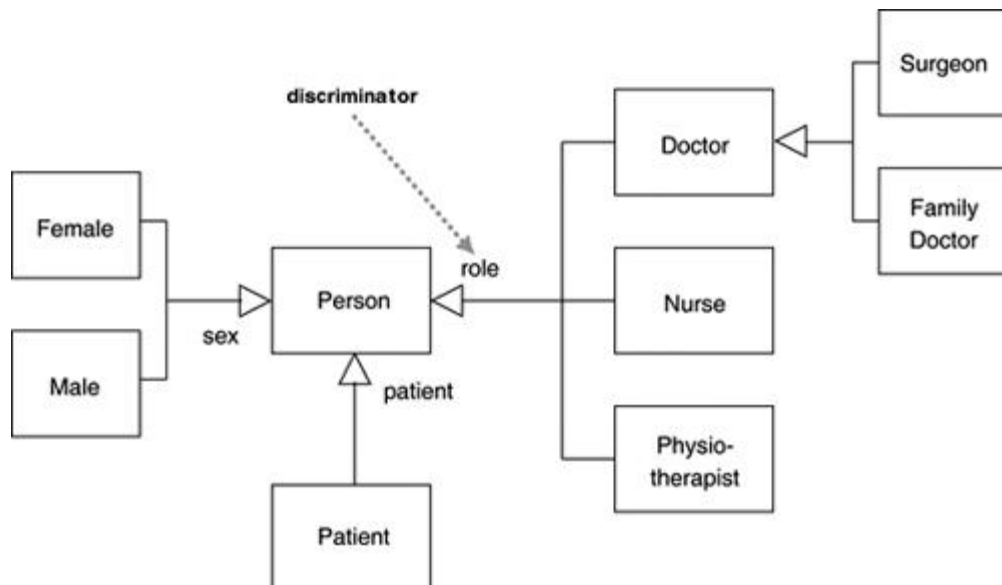
Classification refers to the relationship between an object and its type. Mainstream programming languages assume that an object belongs to a single class. But there are more options to classification than that.

In single classification, an object belongs to a single type, which may inherit from supertypes. In multiple classification, an object may be described by several types that are not necessarily connected by inheritance.

Multiple classification is different from multiple inheritance. Multiple inheritance says that a type may have many supertypes but that a single type must be defined for each object. Multiple classification allows multiple types for an object without defining a specific type for the purpose.

For example, consider a person subtyped as either man or woman, doctor or nurse, patient or not (see [Figure 5.11](#)). Multiple classification allows an object to have any of these types assigned to it in any allowable combination, without the need for types to be defined for all the legal combinations.

Figure 5.11. Multiple classification



If you use multiple classification, you need to be sure that you make it clear which combinations are legal. UML 2 does this by placing each generalization relationship into a generalization set. On the class diagram, you label the generalization arrowhead with the name of the generalization set, which in UML 1 was called the discriminator. Single classification corresponds to a single generalization set with no name.

Generalization sets are by default disjoint: Any instance of the supertype may be an instance of only one of the subtypes within that set. If you roll up generalizations into a single arrow, they must all be part of the same generalization set, as shown in [Figure 5.11](#). Alternatively, you can have several arrows with the same text label.

To illustrate, note the following legal combinations of subtypes in the diagram: (Female, Patient, Nurse); (Male, Physiotherapist); (Female, Patient); and (Female, Doctor, Surgeon). The combination (Patient, Doctor, Nurse) is illegal because it contains two types from the role generalization set.

Another question is whether an object may change its class. For example, when a bank account is overdrawn, it substantially changes its behavior. Specifically, several operations, including "withdraw" and "close," get overridden.

Dynamic classification allows objects to change class within the subtyping structure; static classification does not. With static classification, a separation is made between types and states; dynamic classification combines these notions.

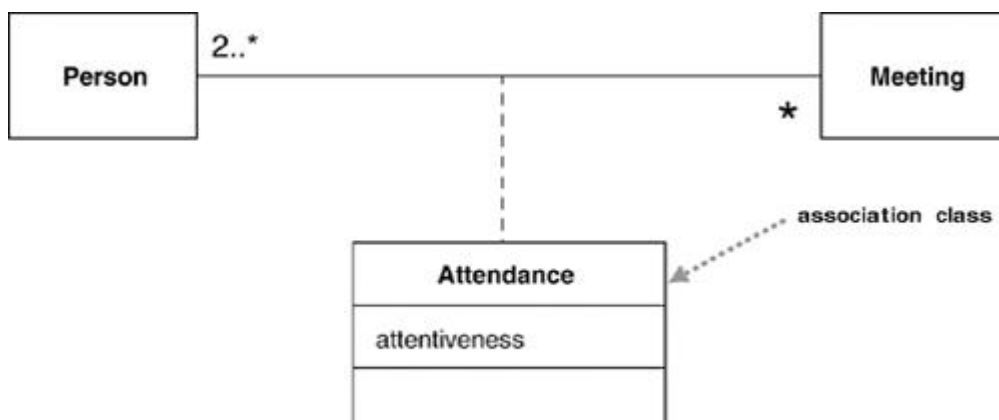
Should you use multiple, dynamic classification? I believe that it is useful for conceptual modeling. For software perspectives, however, the distance between it and the implementations is too much of a leap. In the vast majority of UML diagrams, you'll see only single static classification, so that should be your default.

Association Class

Association classes allow you to add attributes, operations, and other features to associations, as shown in [Figure 5.12](#). We can see from the diagram that a person may attend many meetings. We

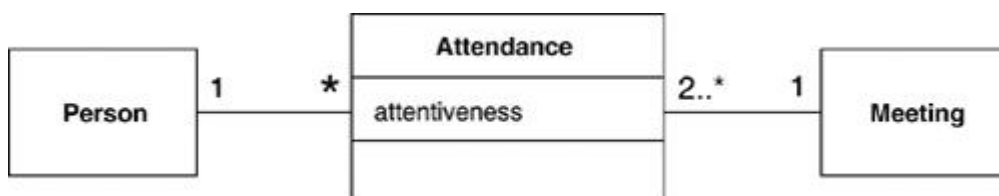
need to keep information about how awake that person was; we can do this by adding the attribute attentiveness to the association.

Figure 5.12. Association class



[Figure 5.13](#) shows another way to represent this information: Make Attendance a full class in its own right. Note how the multiplicities have moved.

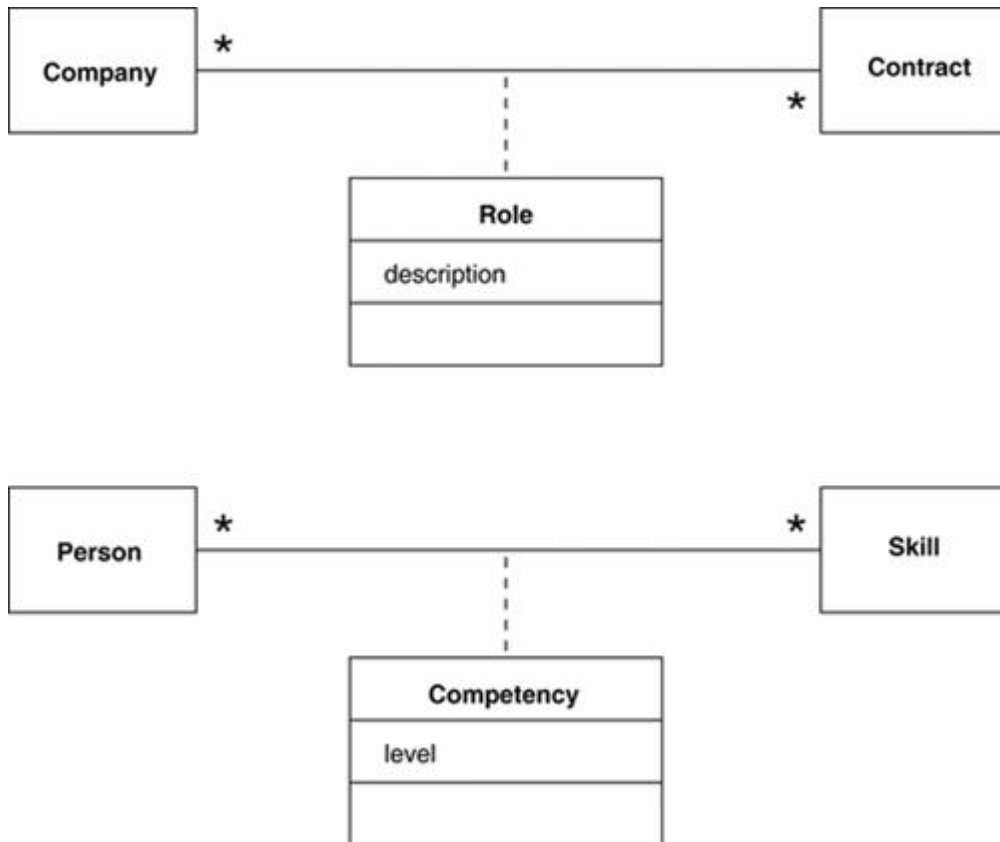
Figure 5.13. Promoting an association class to a full class



What benefit do you gain with the association class to offset the extra notation you have to remember? The association class adds an extra constraint, in that there can be only one instance of the association class between any two participating objects. I feel the need for another example.

Take a look at the two diagrams in [Figure 5.14](#). These diagrams have much the same form. However, we can imagine one **Company** playing different roles in the same **Contract**, but it's harder to imagine a **Person** having multiple competencies in the same skill; indeed, you would probably consider that an error.

Figure 5.14. Association class subtleties (Role should probably not be an association class)



In the UML, only the latter case is legal. You can have only one competency for each combination of Person and Skill. The top diagram in [Figure 5.14](#) would not allow a Company to have more than one Role on a single contract. If you need to allow this, you need to make Role a full class, in the style of [Figure 5.13](#).

Implementing association classes isn't terribly obvious. My advice is to implement an association class as if it were a full class but to provide methods that get information to the classes linked by the association class. So for [Figure 5.12](#), I would see the following methods on Person:

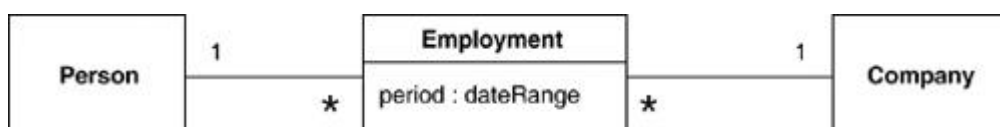
```

class Person
    List getAttendances()
    List getMeetings()
  
```

This way, a client of Person can get hold of the people at the meeting; if they want details, they can get the Attendances themselves. If you do this, remember to enforce the constraint that there can be only one Attendance object for any pair of Person and Meeting. You should place a check in whichever method creates the Attendance.

You often find this kind of construct with historical information, such as in [Figure 5.15](#). However, I find that creating extra classes or association classes can make the model tricky to understand, as well as tilt the implementation in a particular direction that's often unsuitable.

Figure 5.15. Using a class for a temporal relationship



If I have this kind of temporal information, I use a `<<temporal>>` keyword on the association (see [Figure 5.16](#)). The model indicates that a Person may work for only a single Company at one time.

Over time, however, a Person may work for several Companies. This suggests an interface along the lines of:

Figure 5.16. «Temporal» keyword for associations



```
class Person ...
    Company getEmployer(); //get current employer
    Company getEmployer(Date); //get employer at a given date
    void changeEmployer(Company newEmployer, Date changeDate);
    void leaveEmployer (Date changeDate);
```

The «temporal» keyword is not part of the UML, but I mention it here for two reasons. First, it is a notion I have found useful on several occasions in my modeling career. Second, it shows how you can use keywords to extend the UML. You can read a lot more about this at <http://martinfowler.com/ap2/timeNarrative.html>.

Template (Parameterized) Class

Several languages, most noticeably C++, have the notion of a parameterized class, or template. (Templates are on the list to be included in Java and C# in the near future.)

This concept is most obviously useful for working with collections in a strongly typed language. This way, you can define behavior for sets in general by defining a template class `Set`.

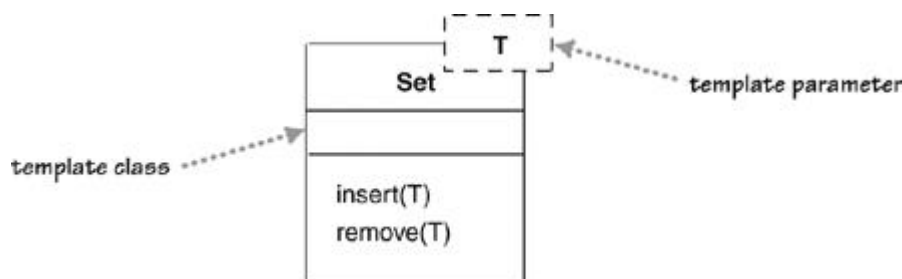
```
class Set <T> {
    void insert (T newElement);
    void remove (T anElement);
```

When you have done this, you can use the general definition to make `Set` classes for more specific elements:

```
Set <Employee> employeeSet;
```

You declare a template class in the UML by using the notation shown in [Figure 5.17](#). The T in the diagram is a placeholder for the type parameter. (You may have more than one.)

Figure 5.17. Template class



A use of a parameterized class, such as `Set<Employee>`, is called a derivation. You can show a derivation in two ways. The first way mirrors the C++ syntax (see [Figure 5.18](#)). You describe the derivation expression within angle brackets in the form `<parameter-name::parameter-value>`. If

there's only one parameter, conventional use often omits the parameter name. The alternative notation (see [Figure 5.19](#)) reinforces the link to the template and allows you to rename the bound element.

Figure 5.18. Bound element (version 1)

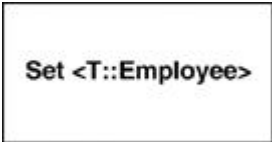
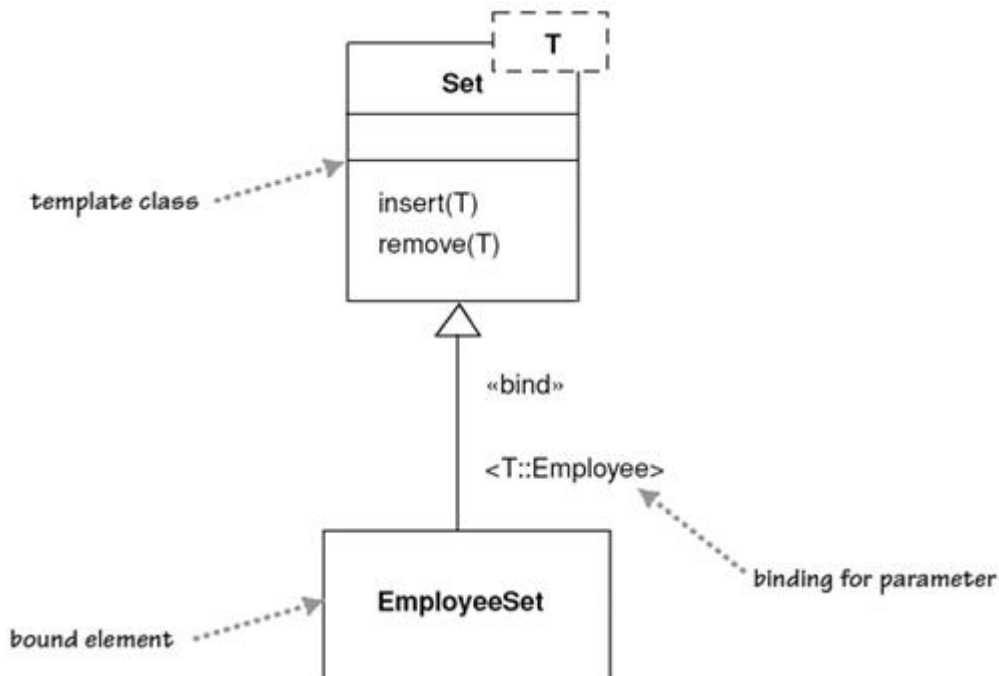


Figure 5.19. Bound element (version 2)



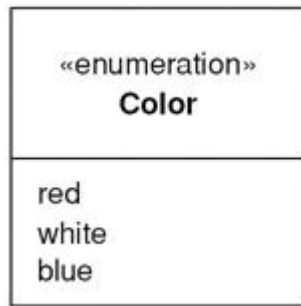
The **«bind»** keyword is a stereotype on the refinement relationship. This relationship indicates that **EmployeeSet** will conform to the interface of **Set**. You can think of the **EmployeeSet** as a subtype of **Set**. This fits the other way of implementing type-specific collections, which is to declare all appropriate subtypes.

Using a derivation is *not* the same as subtyping, however. You are not allowed to add features to the bound element, which is completely specified by its template; you are adding only restricting type information. If you want to add features, you must create a subtype.

Enumerations

Enumerations ([Figure 5.20](#)) are used to show a fixed set of values that don't have any properties other than their symbolic value. They are shown as the class with the **«enumeration»** keyword.

Figure 5.20. Enumeration

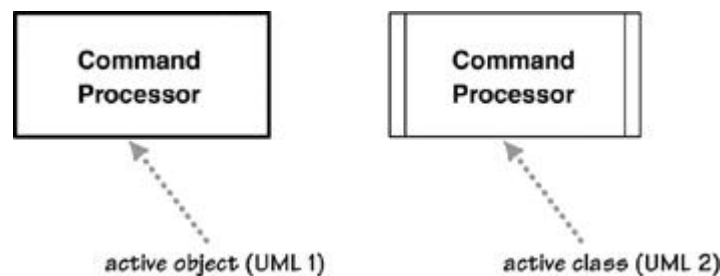


Active Class

An active class has instances, each of which executes and controls its own thread of control. Method invocations may execute in a client's thread or in the active object's thread. A good example of this is a command processor that accepts command objects from the outside and then executes the commands within its own thread of control.

The notation for active classes has changed from UML 1 to UML 2, as shown in [Figure 5.21](#). In UML 2, an active class has extra vertical lines on the side; in UML 1, it had a thick border and was called an active object.

Figure 5.21. Active class



Visibility

Visibility is a subject that is simple in principle but has complex subtleties. The simple idea is that any class has public and private elements. Public elements can be used by any other class; private elements can be used only by the owning class. However, each language makes its own rules. Although many languages use such terms as *public*, *private*, and *protected*, they mean different things in different languages. These differences are small, but they lead to confusion, especially for those of us who use more than one language.

The UML tries to address this without getting into a horrible tangle. Essentially, within the UML, you can tag any attribute or operation with a visibility indicator. You can use any marker you like, and its meaning is language dependent. However, the UML provides four abbreviations for visibility: + (public), - (private), ~ (package), and # (protected). These four levels are used within the UML meta-model and are defined within it, but their definitions vary subtly from those in other languages.

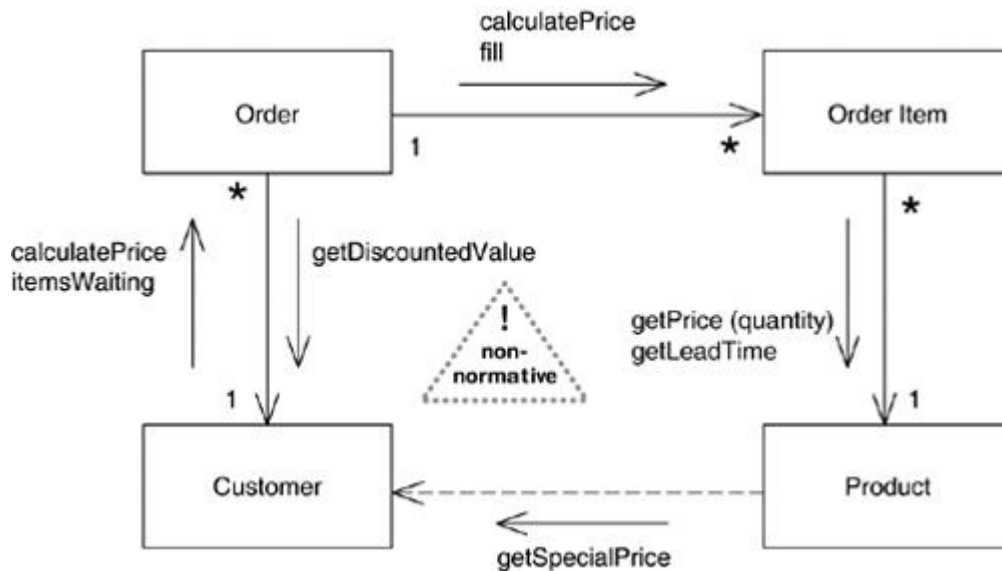
When you are using visibility, use the rules of the language in which you are working. When you are looking at a UML model from elsewhere, be wary of the meanings of the visibility markers, and be aware of how those meanings can change from language to language.

Most of the time, I don't draw visibility markers in diagrams; I use them only if I need to highlight the differences in visibility of certain features. Even then, I can mostly get away with + and –, which at least are easy to remember.

Messages

Standard UML does not show any information about message calls on class diagrams. However, I've sometimes seen conventional diagrams like [Figure 5.22](#).

Figure 5.22. Classes with messages



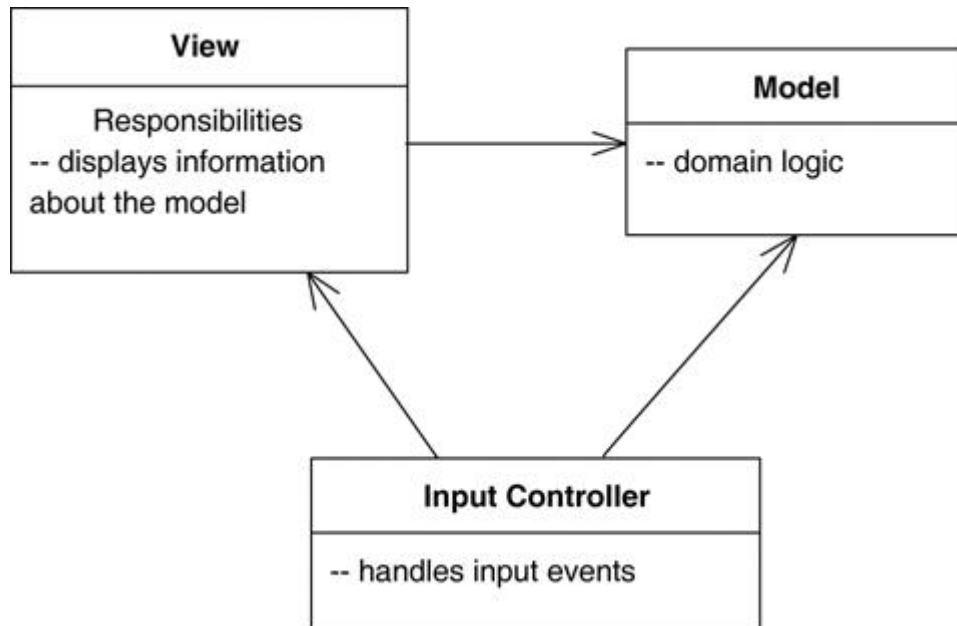
These add arrows to the sides of associations. The arrows are labeled with the messages that one object sends to another. Because you don't need an association to a class to send a message to it, you may also need to add a dependency arrow to show messages between classes that aren't associated.

This message information spans multiple use cases, so they aren't numbered to show sequences, unlike communication diagrams.

Responsibilities

Often, it's handy to show responsibilities (page 63) on a class in a class diagram. The best way to show them is as comment strings in their own compartment in the class ([Figure 5.1](#)). You can name the compartment, if you wish, but I usually don't, as there's rarely any potential for confusion.

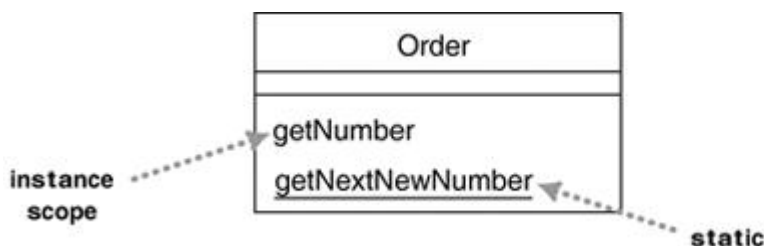
Figure 5.1. Showing responsibilities in a class diagram



Static Operations and Attributes

The UML refers to an operation or an attribute that applies to a class rather than to an instance as static. This is equivalent to static members in C-based languages. Static features are underlined on a class diagram (see [Figure 5.2](#)).

Figure 5.2. Static notation

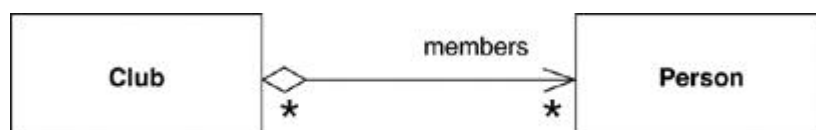


Aggregation and Composition

One of the most frequent sources of confusion in the UML is aggregation and composition. It's easy to explain glibly: Aggregation is the part-of relationship. It's like saying that a car has an engine and wheels as its parts. This sounds good, but the difficult thing is considering what the difference is between aggregation and association.

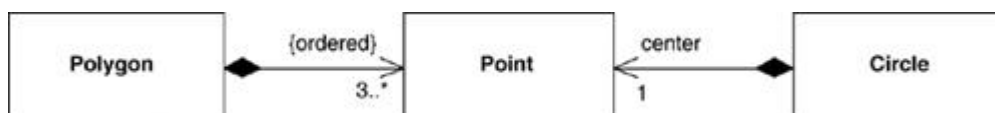
In the pre-UML days, people were usually rather vague on what was aggregation and what was association. Whether vague or not, they were always inconsistent with everyone else. As a result, many modelers think that aggregation is important, although for different reasons. So the UML included aggregation ([Figure 5.3](#)) but with hardly any semantics. As Jim Rumbaugh says, "Think of it as a modeling placebo" [Rumbaugh, UML Reference].

Figure 5.3. Aggregation



As well as aggregation, the UML has the more defined property of composition. In [Figure 5.4](#), an instance of Point may be part of a polygon or may be the center of a circle, but it cannot be both. The general rule is that, although a class may be a component of many other classes, any instance must be a component of only one owner. The class diagram may show multiple classes of potential owners, but any instance has only a single object as its owner.

Figure 5.4. Composition



You'll note that I don't show the reverse multiplicities in [Figure 5.4](#). In most cases, as here, it's 0..1. Its only other possible value is 1, for cases in which the component class is designed so that it can have only one other class as its owner.

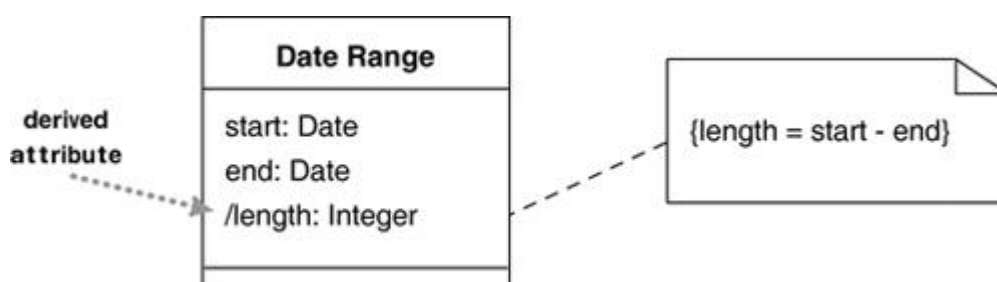
The "no sharing" rule is the key to composition. Another assumption is that if you delete the polygon, it should automatically ensure that any owned Points also are deleted.

Composition is a good way of showing properties that own by value, properties to value objects (page 73), or properties that have a strong and somewhat exclusive ownership of particular other components. Aggregation is strictly meaningless; as a result, I recommend that you ignore it in your own diagrams. If you see it in other people's diagrams, you'll need to dig deeper to find out what they mean by it. Different authors and teams use it for very different purposes.

Derived Properties

Derived properties can be calculated based on other values. When we think about a date range ([Figure 5.5](#)), we can think of three properties: the start date, the end date, and the number of days in the period. These values are linked, so we can think of the length as being derived from the other two values.

Figure 5.5. Derived attribute in a time period



Derivation in software perspectives can be interpreted in a couple of different ways. You can use derivation to indicate the difference between a calculated value and a stored value. In this case, we would interpret [Figure 5.5](#) as indicating that the start and end are stored but that the length is computed. Although this is a common use, I'm not so keen, because it reveals too much of the internals of `DateRange`.

My preferred thinking is that it indicates a constraint between values. In this case, we are saying that the constraint among the three values holds, but it isn't important which of the three values is computed. In this case, the choice of which attribute to mark as derived is arbitrary and strictly unnecessary, but it's useful to help remind people of the constraint. This usage also makes sense with conceptual diagrams.

Derivation can also be applied to properties using association notation. In this case, you simply mark the name with a /.

Interfaces and Abstract Classes

An abstract class is a class that cannot be directly instantiated. Instead, you instantiate an instance of a subclass. Typically, an abstract class has one or more operations that are abstract. An abstract operation has no implementation; it is pure declaration so that clients can bind to the abstract class.

The most common way to indicate an abstract class or operation in the UML is to *italicize* the name. You can also make properties abstract, indicating an abstract property or accessor methods. Italics are tricky to do on a whiteboards, so you can use the label: {abstract}.

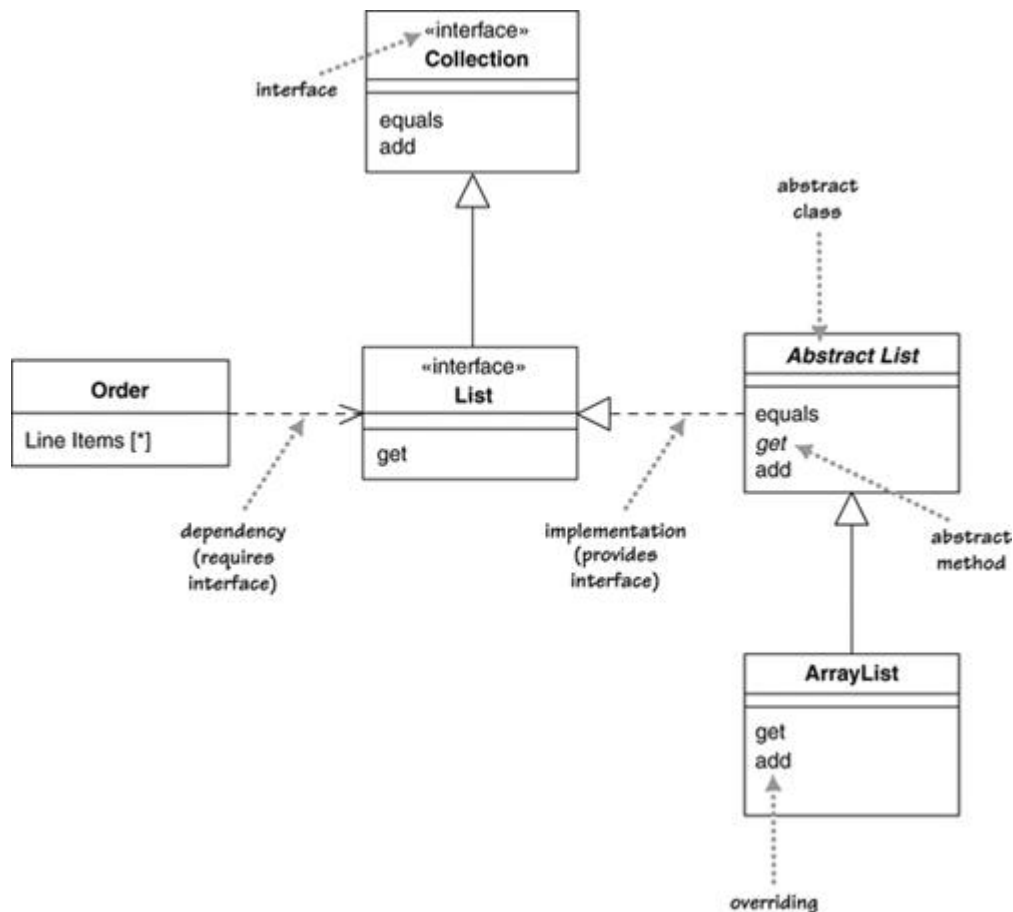
An interface is a class that has no implementation; that is, all its features are abstract. Interfaces correspond directly to interfaces in C# and Java and are a common idiom in other typed languages. You mark an interface with the keyword «interface».

Classes have two kinds of relationships with interfaces: providing and requiring. A class provides an interface if it is substitutable for the interface. In Java and .NET, a class can do that by implementing the interface or implementing a subtype of the interface. In C++, you subclass the class that is the interface.

A class requires an interface if it needs an instance of that interface in order to work. Essentially, this is having a dependency on the interface.

[Figure 5.6](#) shows these relationships in action, based on a few collection classes from Java. I might write an `Order` class that has a list of line items. Because I'm using a list, the `Order` class is dependent on the `List` interface. Let's assume that it uses the methods `equals`, `add`, and `get`. When the objects connect, the `Order` will actually use an instance of `ArrayList` but need not know that in order to use those three methods, as they are all part of the `List` interface.

Figure 5.6. A Java example of interfaces and an abstract class



The `ArrayList` itself is a subclass of the `AbstractList` class. `AbstractList` provides some, but not all, the implementation of the `List` behavior. In particular, the `get` method is abstract. As a result, `ArrayList` implements `get` but also overrides some of the other operations on `AbstractList`. In this case, it overrides `add` but is happy to inherit the implementation of `equals`.

Why don't I simply avoid this and have `Order` use `ArrayList` directly? By using the interface, I allow myself the advantage of making it easier to change implementations later on if I need to. Another implementation may provide performance improvements, some database interaction features, or other benefits. By programming to the interface rather than to the implementation, I avoid having to change all the code should I need a different implementation of `List`. You should always try to program to an interface like this; always use the most general type you can.

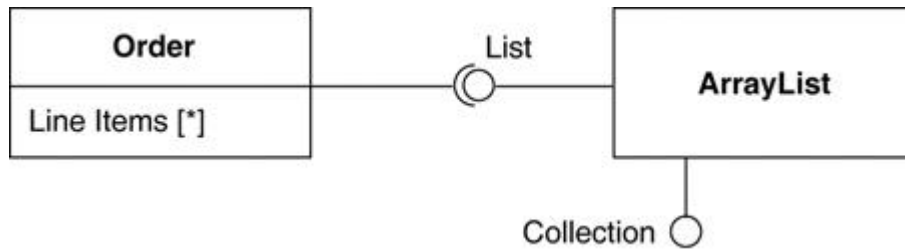
I should also point out a pragmatic wrinkle in this. When programmers use a collection like this, they usually initialize the collection with a declaration, like this:

```
private List lineItems = new ArrayList();
```

Note that this strictly introduces a dependency from `Order` to the concrete `ArrayList`. In theory, this is a problem, but people don't worry about it in practice. Because the type of `lineItems` is declared as `List`, no other part of the `Order` class is dependent on `ArrayList`. Should we change the implementation, there's only this one line of initialization code that we need to worry about. It's quite common to refer to a concrete class once during creation but to use only the interface afterward.

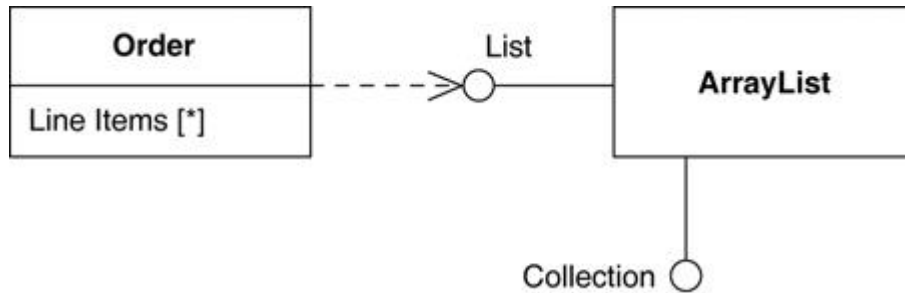
The full notation of [Figure 5.6](#) is one way to notate interfaces. [Figure 5.7](#) shows a more compact notation. The fact that `ArrayList` implements `List` and `Collection` is shown by having ball icons, often referred to as lollipops, out of it. The fact that `Order` requires a `List` interface is shown by the socket icon. The connection is nicely obvious.

Figure 5.7. Ball-and-socket notation



The UML has used the lollipop notation for a while, but the socket notation is new to UML 2. (I think it's my favorite notational addition.) You'll probably see older diagrams use the style of [Figure 5.8](#), where a dependency stands in for the socket notation.

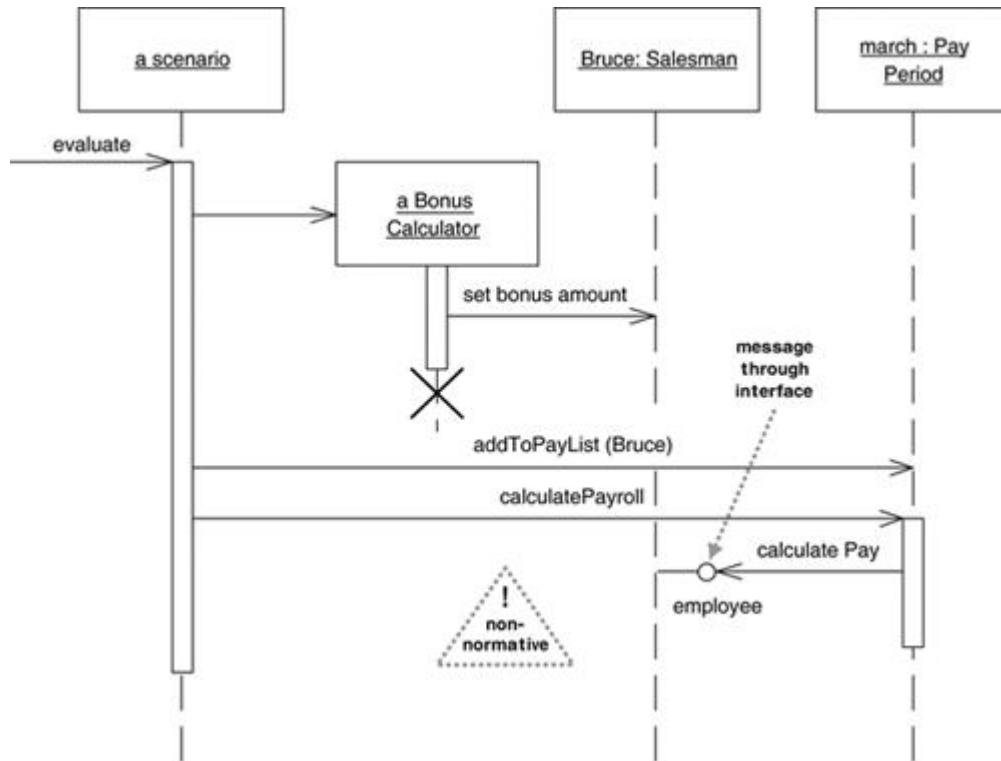
Figure 5.8. Older dependencies with lollipops



Any class is a mix of an interface and an implementation. Therefore, we may often see an object used through the interface of one of its superclasses. Strictly, it wouldn't be legal to use the lollipop notation for a superclass, as the superclass is a class, not a pure interface. But I bend these rules for clarity.

As well as on class diagrams, people have found lollipops useful elsewhere. One of the perennial problems with interaction diagrams is that they don't provide a very good visualization for polymorphic behavior. Although it's not normative usage, you can indicate this along the lines of [Figure 5.9](#). Here, we can see that, although we have an instance of Salesman, which is used as such by the Bonus Calculator, the Pay Period object uses the Salesman only through its Employee interface. (You can do the same trick with communication diagrams.)

Figure 5.9. Using a lollipop to show polymorphism in a sequence diagram



Read-Only and Frozen

On page 37, I described the `{readOnly}` keyword. You use this keyword to mark a property that can only be read by clients and that cannot be updated. Similar yet different is the `{frozen}` keyword from UML 1. A property is frozen if it cannot change during the lifetime of an object; such properties are often called immutable. Although it was dropped from UML 2, `{frozen}` is a very useful concept, so I would continue to use it. As well as marking individual properties as frozen, you can apply the keyword to a class to indicate that all properties of all instances are frozen. (I have heard that frozen may well be reinstated shortly.)

Reference Objects and Value Objects

One of the common things said about objects is that they have identity. This is true, but it is not quite as simple as that. In practice, you find that identity is important for reference objects but not so important for value objects.

Reference objects are such things as Customer. Here, identity is very important because you usually want only one software object to designate a customer in the real world. Any object that references a Customer object will do so through a reference, or pointer; all objects that reference this Customer will reference the same software object. That way, changes to a Customer are available to all users of the Customer.

If you have two references to a Customer and wish to see whether they are the same, you usually compare their identities. Copies may be disallowed; if they are allowed, they tend to be made rarely, perhaps for archive purposes or for replication across a network. If copies are made, you need to sort out how to synchronize changes.

Value objects are such things as Date. You often have multiple value objects representing the same object in the real world. For example, it is normal to have hundreds of objects that designate 1-Jan-04. These are all interchangeable copies. New dates are created and destroyed frequently.

If you have two dates and wish to see whether they are the same, you don't look at their identities but rather at the values they represent. This usually means that you have to write an equality test operator, which for dates would make a test on year, month, and day—or whatever the internal representation is. Each object that references 1-Jan-04 usually has its own dedicated object, but you can also share dates.

Value objects should be immutable; in other words, you should not be able to take a date object of 1-Jan-04 and change the same date object to be 2-Jan-04. Instead, you should create a new 2-Jan-04 object and use that instead. The reason is that if the date were shared, you would update another object's date in an unpredictable way, a problem referred to as aliasing.

In days gone by, the difference between reference objects and value objects was clearer. Value objects were the built-in values of the type system. Now you can extend the type system with your own classes, so this issue requires more thought.

The UML uses the concept of data type, which is shown as a keyword on the class symbol. Strictly, data type isn't the same as value object, as data types can't have identity. Value objects may have an identity, but don't use it for equality. Primitives in Java would be data types, but dates would not, although they would be value objects.

If it's important to highlight them, I use composition when associating with a value object. You can also use a keyword on a value type; common conventional ones I see are «value» or «struct».

Qualified Associations

A qualified association is the UML equivalent of a programming concept variously known as associative arrays, maps, hashes, and dictionaries. [Figure 5.10](#) shows a way that uses a qualifier to represent the association between the Order and Order Line classes. The qualifier says that in connection with an Order, there may be one Order Line for each instance of Product.

Figure 5.10. Qualified association



From a software perspective, this qualified association would imply an interface along the lines of

```
class Order ...
    public OrderLine getLineItem(Product aProduct);
    public void addLineItem(Number amount, Product forProduct);
```

Thus, all access to a given Order Line requires a Product as an argument, suggesting an implementation using a key and value data structure.

It's common for people to get confused about the multiplicities of a qualified association. In [Figure 5.10](#), an Order may have many Line Items, but the multiplicity of the qualified association is the multiplicity in the context of the qualifier. So the diagram says that an Order has 0..1 Line Items per Product. A multiplicity of 1 would indicate that Order would have to have a Line Item for every instance of Product. A * would indicate that you would have multiple Line Items per Product but that access to the Line Items is indexed by Product.

In conceptual modeling, I use the qualifier construct only to show constraints along the lines of "single Order Line per Product on Order."

Chapter 6. Object Diagrams

An object diagram is a snapshot of the objects in a system at a point in time. Because it shows instances rather than classes, an object diagram is often called an instance diagram.

You can use an object diagram to show an example configuration of objects. (See [Figure 6.1](#), which shows a set of classes, and [Figure 6.2](#), which shows an associated set of objects.) This latter use is very useful when the possible connections between objects are complicated.

Figure 6.1. Class diagram of Party composition structure

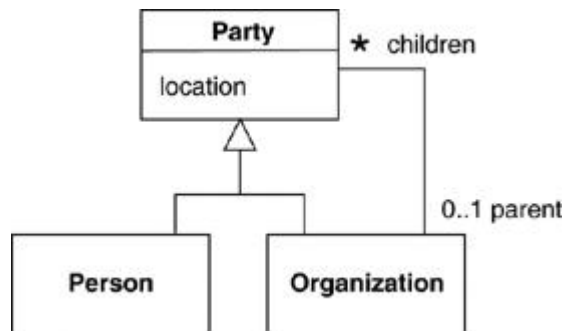
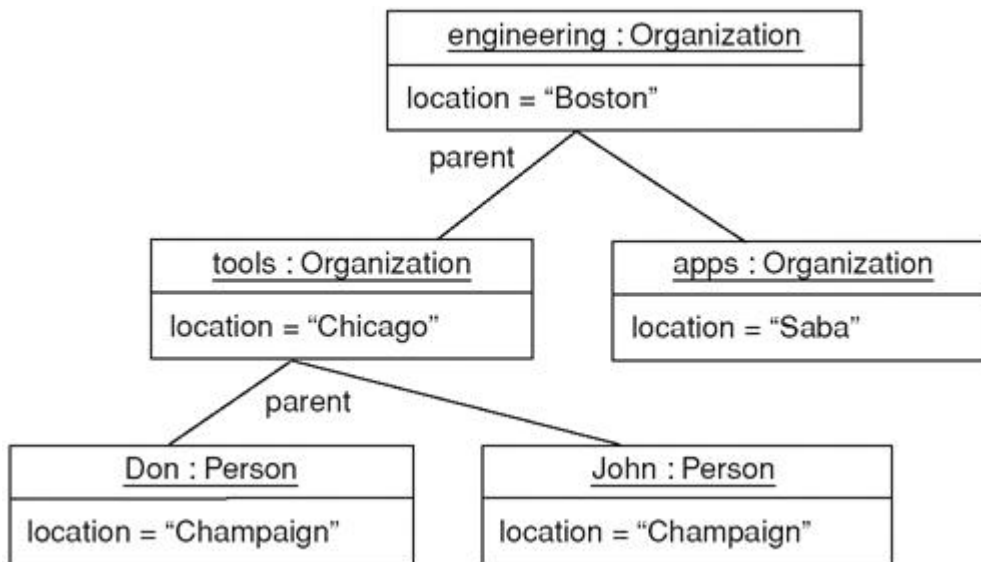


Figure 6.2. Object diagram showing example instances of Party



You can tell that the elements in [Figure 6.2](#) are instances because the names are underlined. Each name takes the form instance name : class name. Both parts of the name are optional, so John, :Person, and aPerson are legal names. If you use only the class name, you must include the colon. You can show values for attributes and links, as in [Figure 6.2](#).

Strictly, the elements of an object diagram are instance specifications rather than true instances. The reason is that it's legal to leave mandatory attributes empty or to show instance specifications of abstract classes. You can think of an instance specification as a partly defined instance.

Another way of looking at an object diagram is as a communication diagram (page 131) without messages.

When to Use Object Diagrams

Object diagrams are useful for showing examples of objects connected together. In many situations, you can define a structure precisely with a class diagram, but the structure is still difficult to understand. In these situations, a couple of object diagram examples can make all the difference.

Chapter 7. Package Diagrams

Classes represent the basic form of structuring an object-oriented system. Although they are wonderfully useful, you need something more to structure large systems, which may have hundreds of classes.

A package is a grouping construct that allows you to take any construct in the UML and group its elements together into higher-level units. Its most common use is to group classes, and that's the way I'm describing it here, but remember that you can use packages for every other bit of the UML as well.

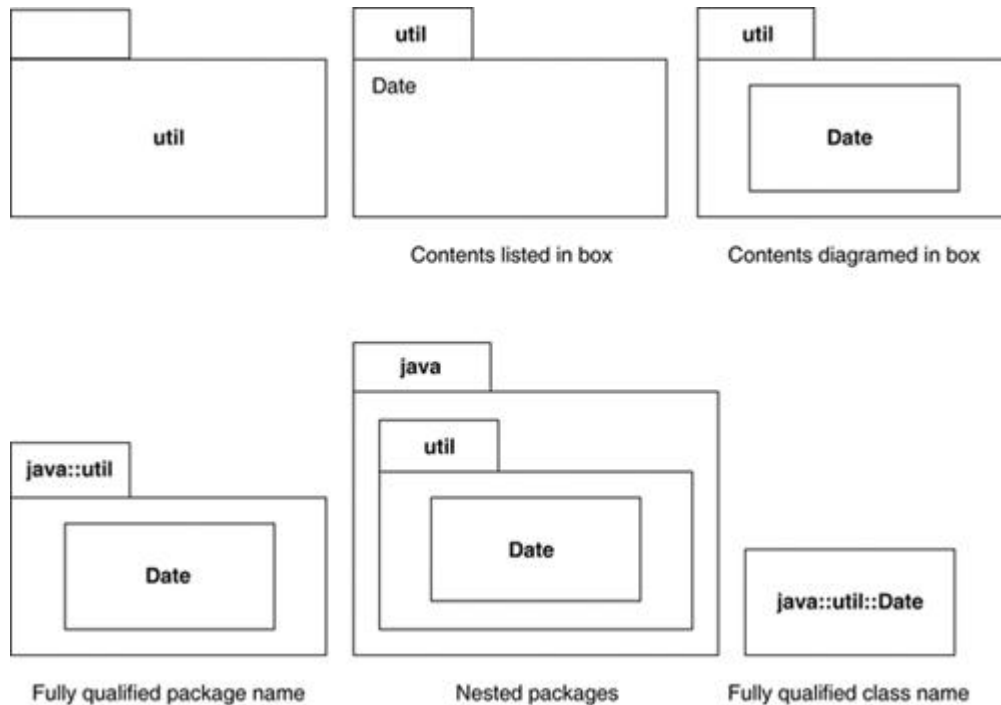
In a UML model, each class is a member of a single package. Packages can also be members of other packages, so you are left with a hierarchic structure in which top-level packages get broken down into subpackages with their own subpackages and so on until the hierarchy bottoms out in classes. A package can contain both subpackages and classes.

In programming terms, packages correspond to such grouping constructs as packages (in Java) and namespaces (in C++ and .NET).

Each package represents a namespace, which means that every class must have a unique name within its owning package. If I want to create a class called `Date`, and a `Date` class is already in the `System` package, I can have my `Date` class as long as I put it in a separate package. To make it clear which is which, I can use a fully qualified name, that is, a name that shows the owning package structure. You use double colons to show package names in UML, so the dates might be `System::Date` and `MartinFowler::Util::Date`.

In diagrams, packages are shown with a tabbed folder, as in [Figure 7.1](#). You can simply show the package name or show the contents too. At any point, you can use fully qualified names or simply regular names. Showing the contents with class icons allows you to show all the details of a class, even to the point of showing a class diagram within the package. Simply listing the names makes sense when all you want to do is indicate which classes are in which packages.

Figure 7.1. Ways of showing packages on diagrams



It's quite common to see a class labeled something like `Date` (from `java.util`) rather than the fully qualified form. This style is a convention that was done a lot by Rational Rose; it isn't part of the standard.

The UML allows classes in a package to be public or private. A public class is part of the interface of the package and can be used by classes in other packages; a private class is hidden. Different programming environments have different rules about visibility between their packaging constructs; you should follow the convention of your programming environment, even if it means bending the UML's rules.

A useful technique here is to reduce the interface of the package by exporting only a small subset of the operations associated with the package's public classes. You can do this by giving all classes private visibility, so that they can be seen only by other classes in the same package, and by adding extra public classes for the public behavior. These extra classes, called *Facades* [Gang of Four], then delegate public operations to their shyer companions in the package.

How do you choose which classes to put in which packages? This is actually quite an involved question that needs a good bit of design skill to answer. Two useful principles are the Common Closure Principle and Common Reuse Principle [Martin]. The Common Closure Principle says that the classes in a package should need changing for similar reasons. The Common Reuse Principle says that classes in a package should all be reused together. Many of the reasons for grouping classes in packages have to do with the dependencies between the packages, which I'll come to next.

Packages and Dependencies

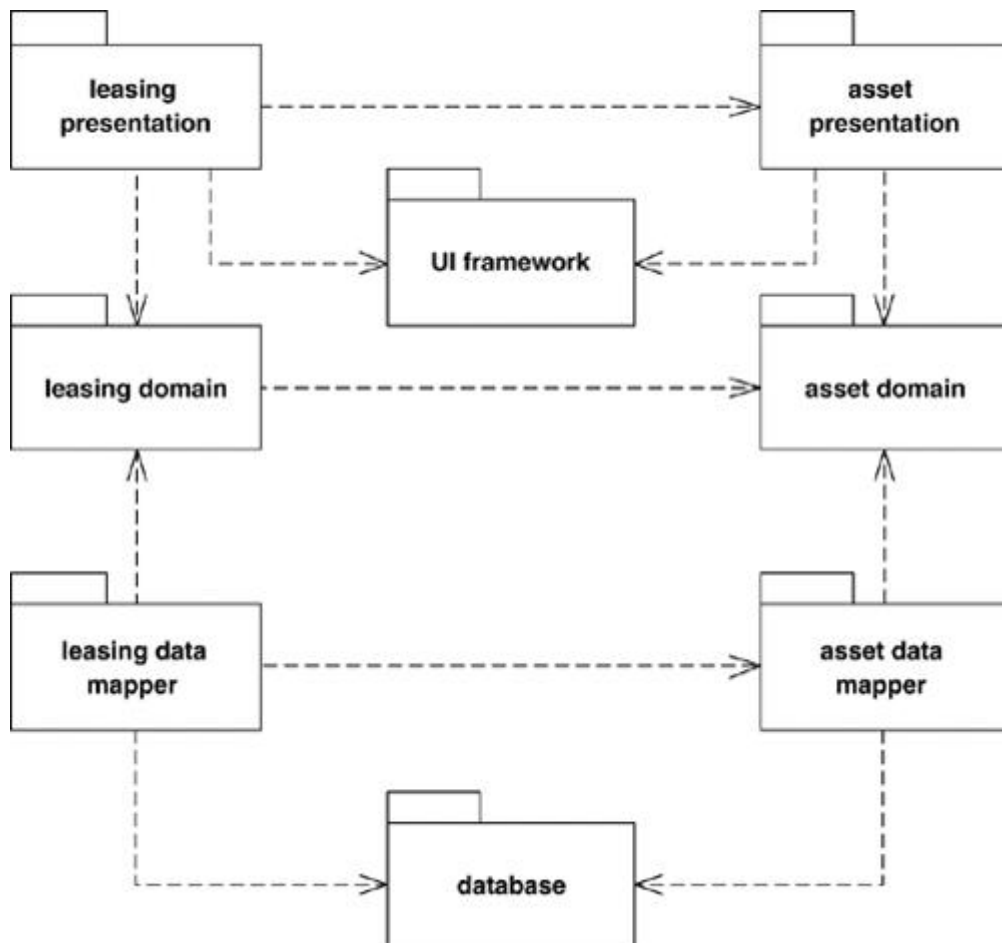
A package diagram shows packages and their dependencies. I introduced the concept of dependency on page 47. If you have packages for presentation and domain, you have a dependency from the presentation package to the domain package if any class in the presentation package has a dependency to any class in the domain package. In this way, interpackage dependencies summarize the dependencies between their contents.

The UML has many varieties of dependency, each with particular semantics and stereotype. I find it easier to begin with the unsteriotyped dependency and use the more particular dependencies only if I need to, which I hardly ever do.

In a medium to large system, plotting a package diagram can be one of the most valuable things you can do to control the large-scale structure of the system. Ideally, this diagram should be generated from the code base itself, so that you can see what is really there in the system.

A good package structure has a clear flow to the dependencies, a concept that's difficult to define but often easier to recognize. [Figure 7.2](#) shows a sample package diagram for an enterprise application, one that is well-structured and has a clear flow.

Figure 7.2. Package diagram for an enterprise application



Often, you can identify a clear flow because all the dependencies run in a single direction. Although that is a good indicator of a well-structured system, the data mapper packages of [Figure 7.2](#) show an exception to that rule of thumb. The data mapper packages act as an insulating layer between the domain and database packages, an example of the Mapper pattern [Fowler, P of EAA].

Many authors say that there should be no cycles in the dependencies (the Acyclic Dependency Principle [Martin]). I don't treat that as an absolute rule, but I do think that cycles should be localized and that, in particular, you shouldn't have cycles that cross layers.

The more dependencies coming into a package, the more stable the package's interface needs to be, as any change in its interface will ripple into all the packages that are dependent on it (the Stable Dependencies Principle [Martin]). So in [Figure 7.2](#), the asset domain package needs a more stable interface than the leasing data mapper package. Often, you'll find that the more stable packages tend to have a higher proportion of interfaces and abstract classes (the Stable Abstractions Principle [Martin]).

The dependency relationships are not transitive (page 48). To see why this is important for dependencies, look at [Figure 7.2](#) again. If a class in the asset domain package changes, we may have a change to classes within the leasing domain package. But this change does not necessarily ripple through to the leasing presentation. (It ripples only if the leasing domain changes its interface.)

Some packages are used in so many places that it would be a mess to draw all the dependency lines to them. In this case, a convention is to use a keyword, such as «global», on the package.

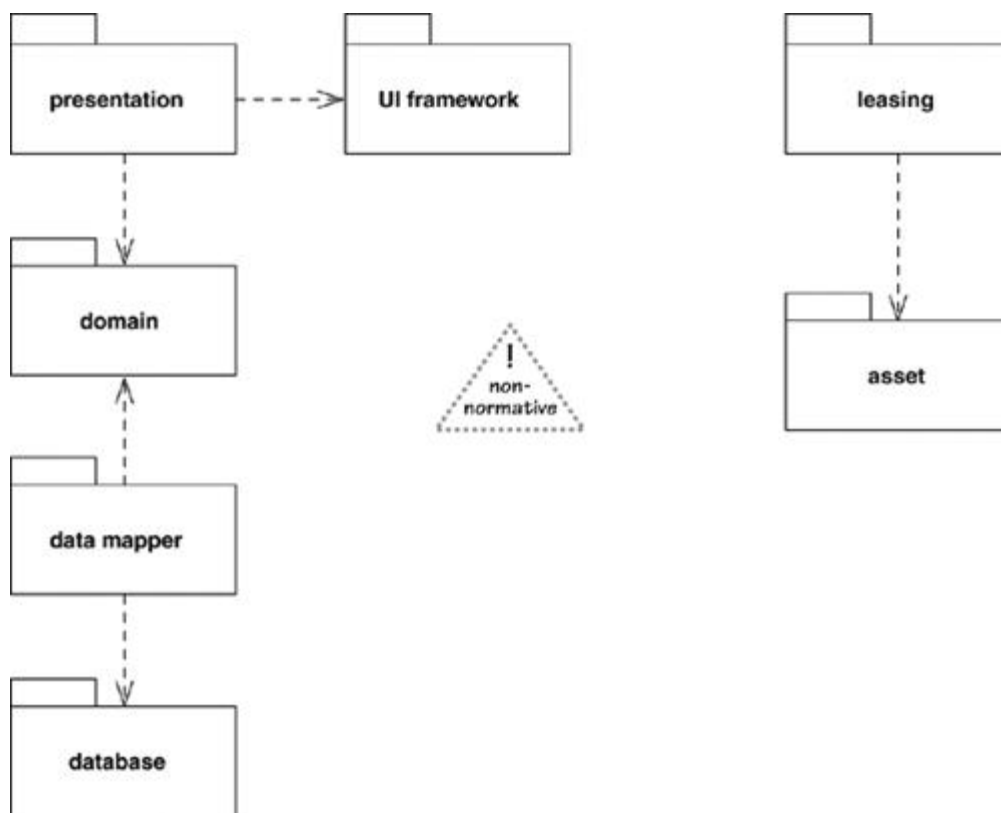
UML packages also define constructs to allow packages to import and merge classes from one package into another, using dependencies with keywords to notate this. However, rules for this kind of thing vary greatly with programming languages. On the whole, I find the general notion of dependencies to be far more useful in practice.

Package Aspects

If you think about [Figure 7.2](#), you'll realize that the diagram has two kinds of structures. One is a structure of layers in the application: presentation, domain, data mapper, and database. The other is a structure of subject areas: leasing and assets.

You can make this more apparent by separating the two aspects, as in [Figure 7.3](#). With this diagram, you can clearly see each aspect. However, these two aspects aren't true packages, because you can't assign classes to a single package. (You would have to pick one from each aspect.) This problem mirrors the problem in the hierarchic namespaces in programming languages. Although diagrams like [Figure 7.3](#) are nonstandard UML, they are often very helpful in explaining the structure of a complex application.

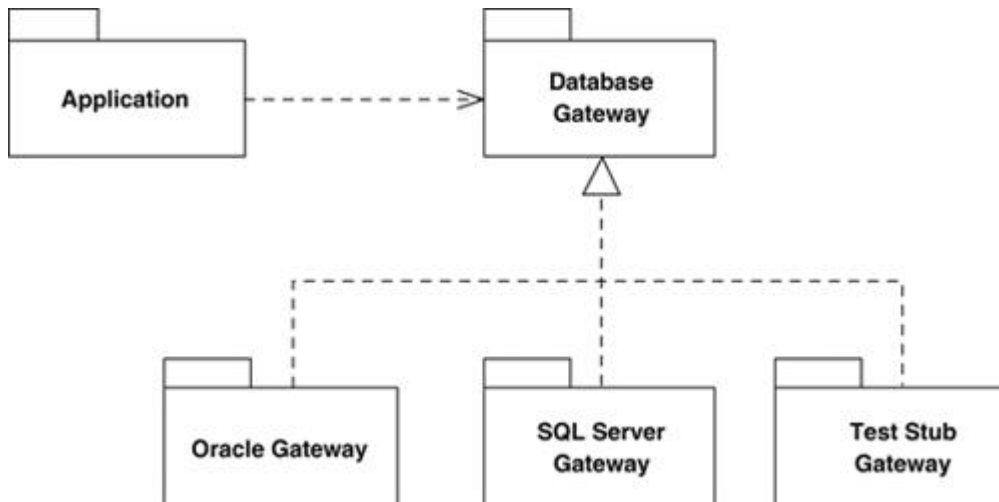
Figure 7.3. Separating [Figure 7.2](#) into two aspects



Implementing Packages

Often, you'll see a case in which one package defines an interface that can be implemented by a number of other packages, such as that of [Figure 7.4](#). In this case, the realization relationship indicates that the database gateway defines an interface and that the other gateway classes provide an implementation. In practice, this would mean that the database gateway package contains interfaces and abstract classes that are fully implemented by the other packages.

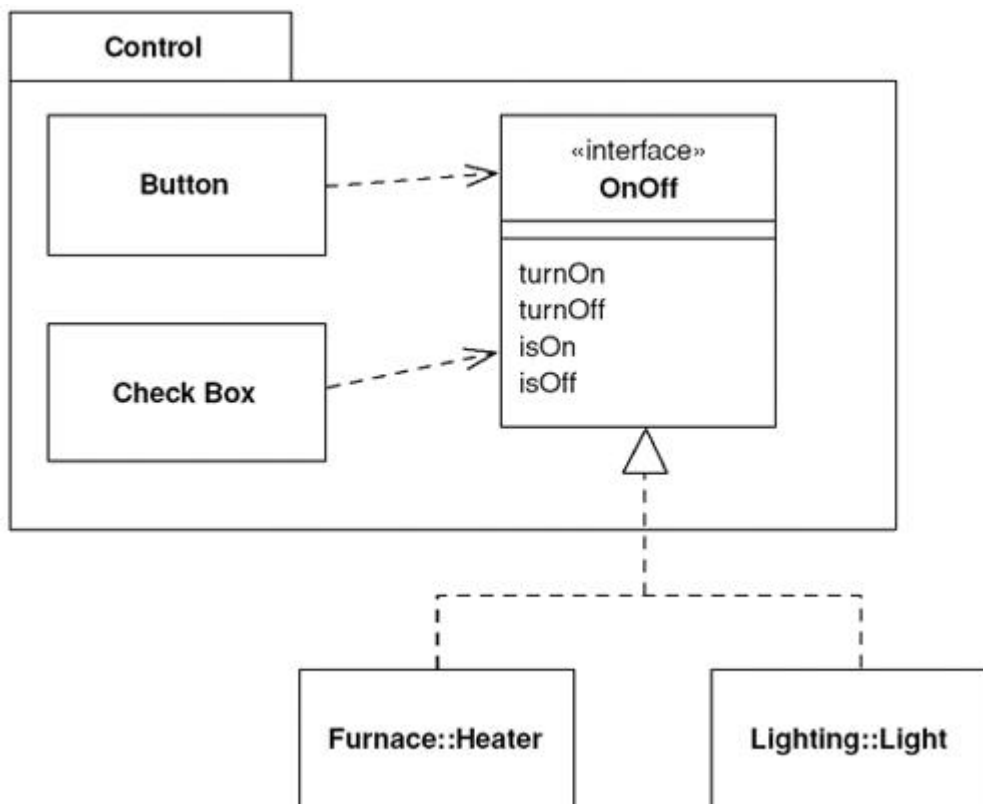
Figure 7.4. A package implemented by other packages



It's quite common for an interface and its implementation to be in separate packages. Indeed, a client package often contains an interface for another package to implement: the same notion of required interface that I discussed on page 70.

Imagine that we want to provide some user interface (UI) controls to turn things on and off. We want this to work with a lot of different things, such as heaters and lights. The UI controls need to invoke methods on the heater, but we don't want the controls to have a dependency to the heater. We can avoid this dependency by defining in the controls package an interface that is then implemented by any class that wants to work with these controls, as in [Figure 7.5](#). This is an example of the pattern Separated Interface [Fowler, P of EAA].

Figure 7.5. Defining a required interface in a client package



When to Use Package Diagrams

I find package diagrams extremely useful on larger-scale systems to get a picture of the dependencies between major elements of a system. These diagrams correspond well to common programming structures. Plotting diagrams of packages and dependencies helps you keep an application's dependencies under control.

Package diagrams represent a compile-time grouping mechanism. For showing how objects are composed at runtime, use a composite structure diagram (page 135).

Where to Find Out More

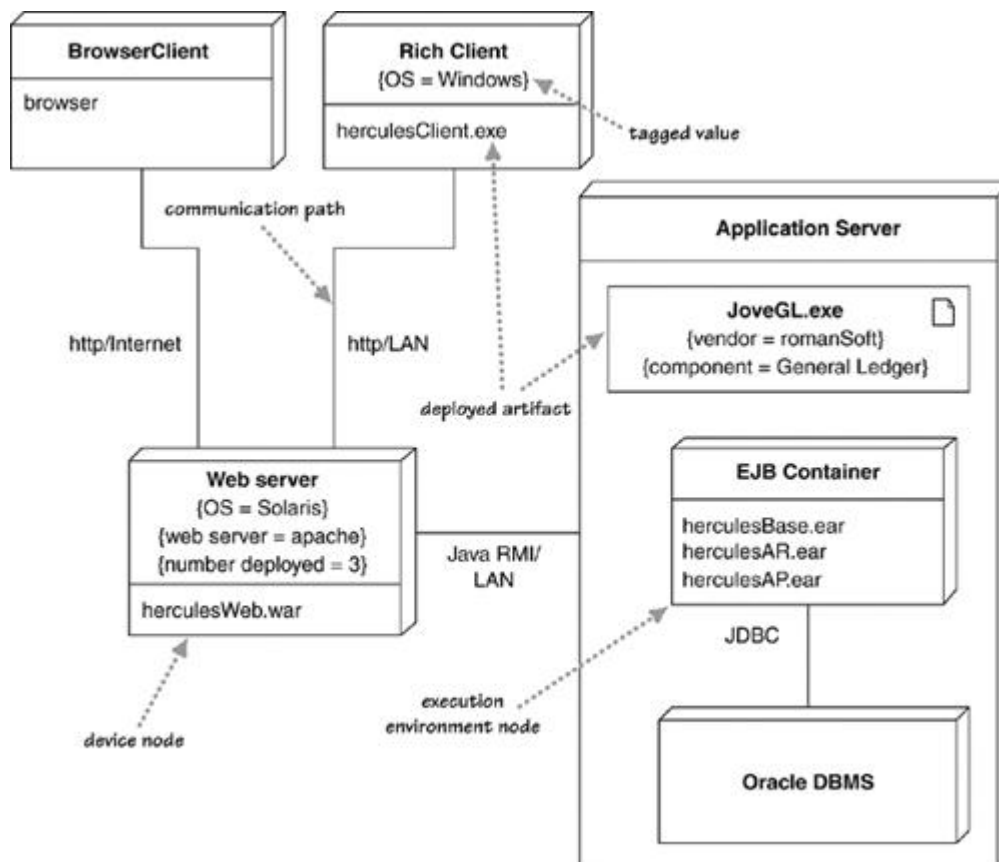
The best discussion I know of packages and how to use them is [Martin]. Robert Martin has long had an almost pathological obsession with dependencies and writes well about how to pay attention to dependencies so that you can control and minimize them.

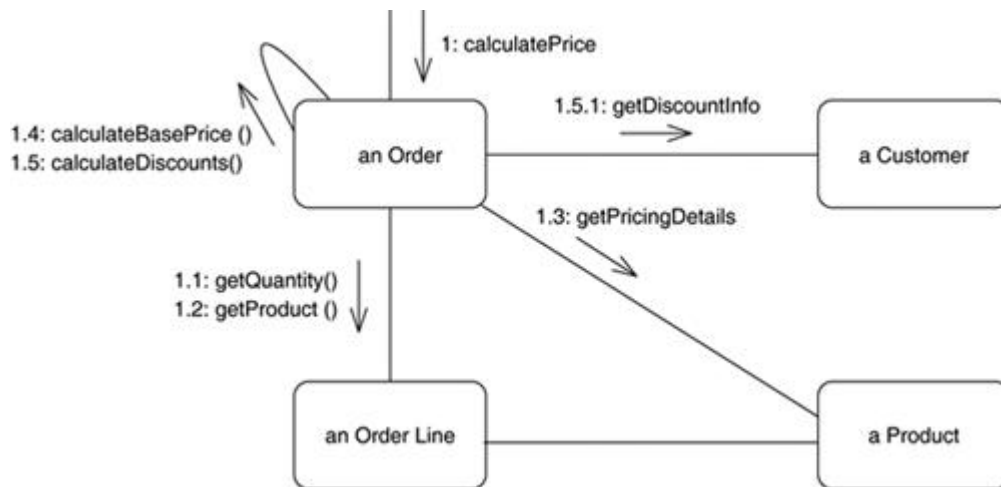
Chapter 8. Deployment Diagrams

Deployment diagrams show a system's physical layout, revealing which pieces of software run on what pieces of hardware. Deployment diagrams are really very simple; hence the short chapter.

[Figure 8.1](#) is a simple example of a deployment diagram. The main items on the diagram are nodes connected by communication paths. A node is something that can host some software. Nodes come in two forms. A device is hardware, it may be a computer or a simpler piece of hardware connected to a system. An execution environment is software that itself hosts or contains other software, examples are an operating system or a container process.

Figure 8.1. Example deployment diagram





The reason for the nested decimal numbers is to resolve ambiguity with self-calls. In [Figure 4.2](#), you can clearly see that `getDiscountInfo` is called within the method `calculateDiscount`. With the flat numbering of [Figure 12.1](#), however, you can't tell whether `getDiscountInfo` is called within `calculateDiscount` or within the overall `calculatePrice` method. The nested numbering scheme resolves this problem.

Despite its illegality, many people prefer a flat numbering scheme. The nested numbers can get very tangled, particularly as calls get rather nested, leading to such sequence numbers as 1.1.1.2.1.1. In these cases, the cure for ambiguity can be worse than the disease.

As well as numbers, you may also see letters on messages; these letters indicate different threads of control. So messages A5 and B2 would be in different threads; messages 1a1 and 1b1 would be different threads concurrently nested within message 1. You also see thread letters on sequence diagrams, although this doesn't convey the concurrency visually.

Communication diagrams don't have any precise notation for control logic. They do allow you to use iteration markers and guards (page 59), but they don't allow you to fully specify control logic. There is no special notation for creating or deleting objects, but the `«create»` and `«delete»` keywords are common conventions.

When to Use Communication Diagrams

The main question with communication diagrams is when to use them rather than the more common sequence diagrams. A strong part of the decision is personal preference: Some people like one over the other. Often, that drives the choice more than anything else. On the whole, most people seem to prefer sequence diagrams, and for once, I'm with the majority.

A more rational approach says that sequence diagrams are better when you want to emphasize the sequence of calls and that communication diagrams are better when you want to emphasize the links. Many people find that communication diagrams are easier to alter on a whiteboard, so they are a good approach for exploring alternatives, although in those cases, I often prefer CRC cards.

Chapter 13. Composite Structures

One of the most significant new features in UML 2 is the ability to hierarchically decompose a class into an internal structure. This allows you to take a complex object and break it down into parts.

[Figure 13.1](#) shows a TV Viewer class with its provided and required interfaces (page 69). I've shown this in two ways: using the ball-and-socket notation and listing them internally.

Figure 13.1. Two ways of showing a TV viewer and its interfaces

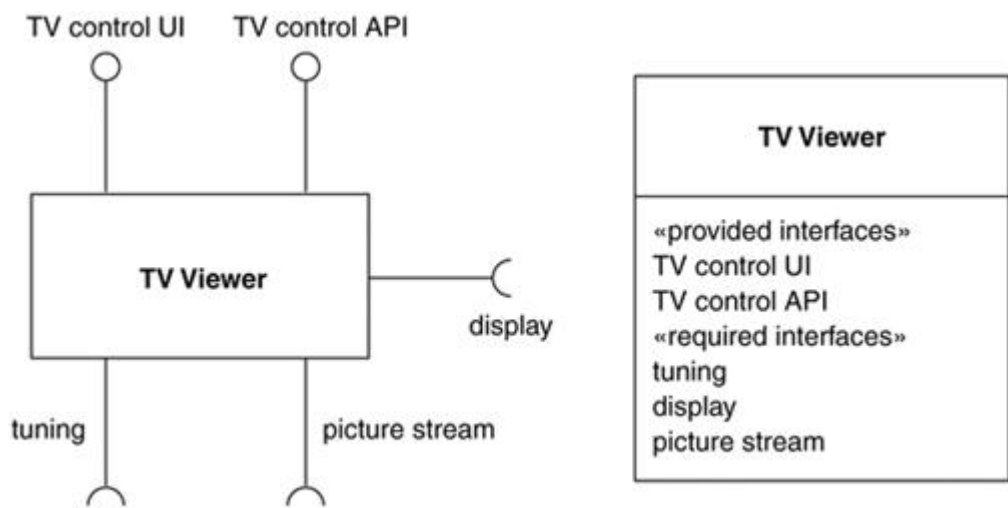
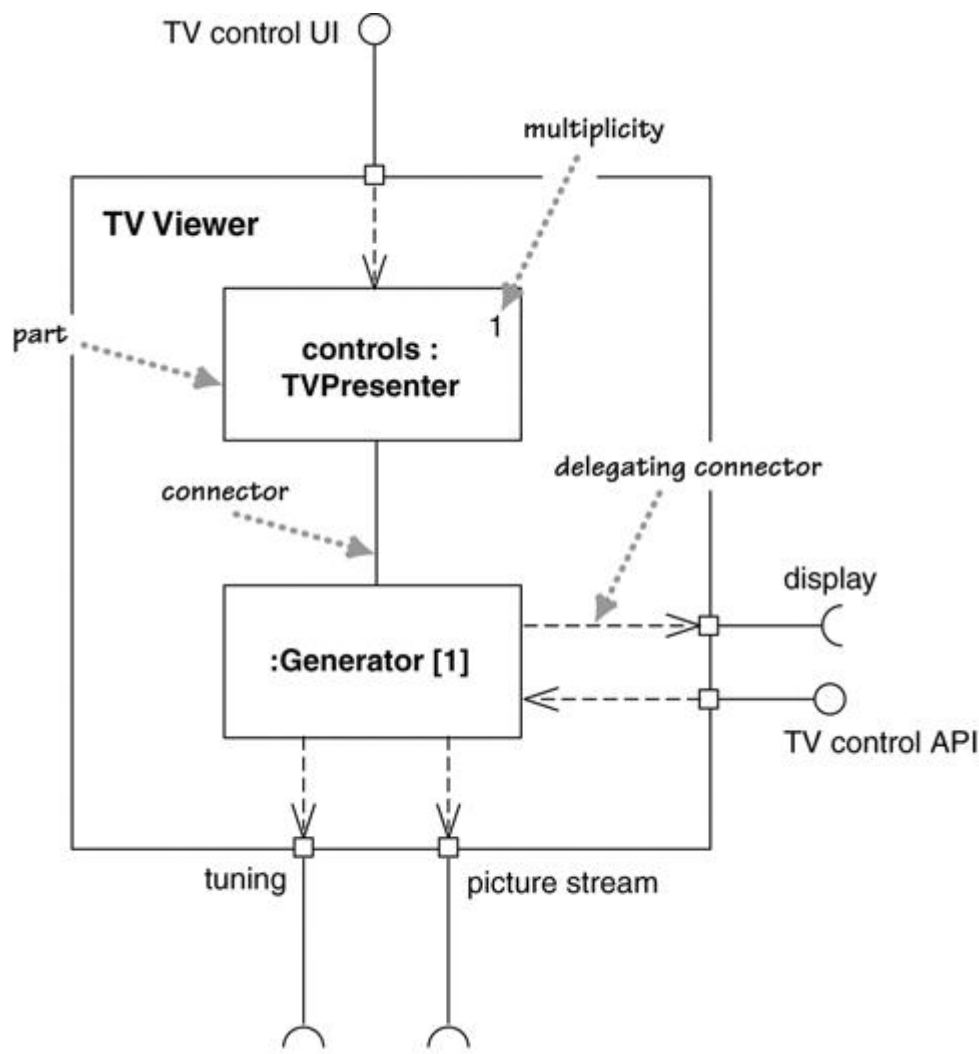


Figure 13.2 shows how this class is decomposed internally into two parts and which parts support and require the different interfaces. Each part is named in the form **name : class**, with both elements individually optional. Parts are not instance specifications, so they are **bolded** rather than underlined.

Figure 13.2. Internal view of a component (example suggested by Jim Rumbaugh)

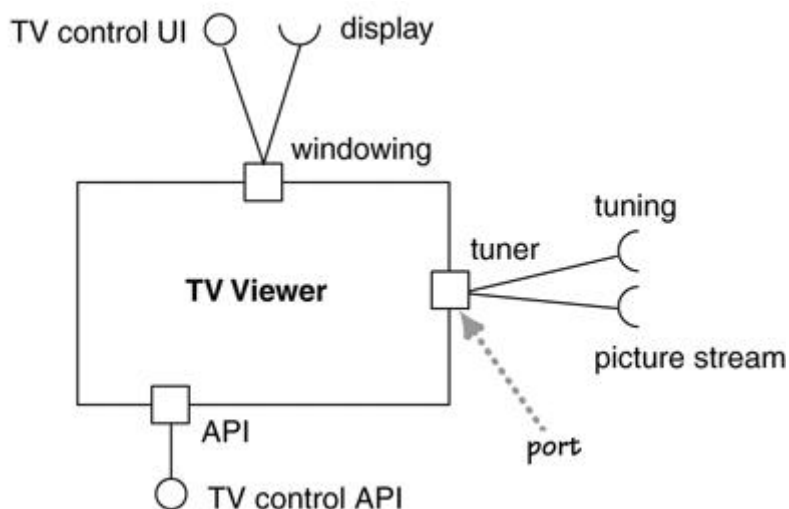


You can show how many instances of a part are present. [Figure 13.2](#) says that each TV Viewer contains one generator part and one controls part.

To show a part implementing an interface, you draw a delegating connector from that interface. Similarly, to show that a part needs an interface, you show a delegating connector to that interface. You can also show connectors between parts with either a simple line, as I've done here, or with ball-and-socket notation (page 71).

You can add ports (Figure 13.3) to the external structure. Ports allow you to group the required and provided interfaces into logical interactions that a component has with the outside world.

Figure 13.3. A component with multiple ports



When to Use Composite Structures

Composite structures are new to UML 2, although some older methods had some similar ideas. A good way of thinking about the difference between packages and composite structures is that packages are a compile-time grouping, while composite structures show runtime groupings. As such, they are a natural fit for showing components and how they are broken into parts; hence, much of this notation is used in component diagrams.

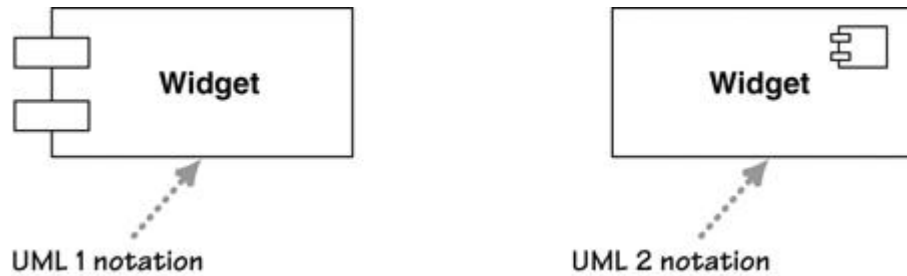
Because composite structures are new to the UML, it's too early to tell how effective they will turn out in practice; many members of the UML committee think that these diagrams will become a very valuable addition.

Chapter 14. Component Diagrams

A debate that's always ranged large in the OO community is what the difference is between a component and any regular class. This is not a debate that I want to settle here, but I can show you the notation the UML uses to distinguish between them.

UML 1 had a distinctive symbol for a component ([Figure 14.1](#)). UML 2 removed that icon but allows you to annotate a class box with a similar-looking icon. Alternatively, you can use the `«component»` keyword.

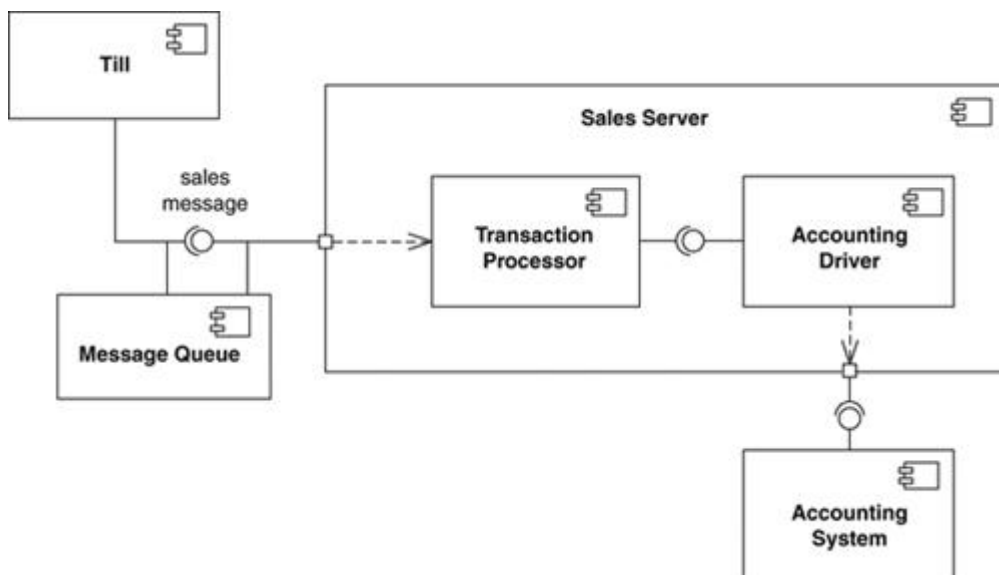
Figure 14.1. Notation for components



Other than the icon, components don't introduce any notation that we haven't already seen. Components are connected through implemented and required interfaces, often using the ball-and-socket notation (page 71) just as for class diagrams. You can also decompose components by using composite structure diagrams.

Figure 14.2 shows an example component diagram. In this example, a sales till can connect to a sales server component, using a sales message interface. Because the network is unreliable, a message queue component is set up so the till can talk to the server when the network is up and talk to a queue when the network is down; the queue will then talk to the server when the network becomes available. As a result, the message queue both supplies the sales message interface to talk with the till and requires that interface to talk with the server. The server is broken down into two major components. The transaction processor realizes the sales message interface, and the accounting driver talks to the accounting system.

Figure 14.2. An example component diagram



As I've already said, the issue of what is a component is the subject of endless debate. One of the more helpful statements I've found is this:

Components are not a technology. Technology people seem to find this hard to understand. Components are about how customers want to relate to software. They want to be able to buy their software a piece at a time, and to be able to upgrade it just like they can upgrade their stereo. They want new pieces to work seamlessly with their old pieces, and to be able to upgrade on their own schedule, not the manufacturer's schedule. They want to be able to mix and match pieces from various manufacturers. This is a very reasonable requirement. It is just hard to satisfy.

Ralph Johnson, <http://www.c2.com/cgi/wiki?DoComponentsExist>

The important point is that components represent pieces that are independently purchasable and upgradeable. As a result, dividing a system into components is as much a marketing decision as it is a technical decision, for which [Hohmann] is an excellent guide. It's also a reminder to beware of

overly fine-grained components, because too many components are hard to manage, especially when versioning rears its ugly head, hence "DLL hell."

In earlier versions of the UML, components were used to represent physical structures, such as DLLs. That's no longer true; for this task, you now use artifacts (page 97).

When to Use Component Diagrams

Use component diagrams when you are dividing your system into components and want to show their interrelationships through interfaces or the breakdown of components into a lower-level structure.

Chapter 15. Collaborations

Unlike the other chapters in this book, this one does not correspond to an official diagram in UML 2. The standard discusses collaborations as part of composite structures, but the diagram is really quite different and was used in UML 1 without any link to composite structures. So I felt it best to discuss collaborations as their own chapter.

Let's consider the notion of an auction. In any auction, we might have a seller, some buyers, a lot of goods, and some offers for the sale. We can describe these elements in terms of a class diagram ([Figure 15.1](#)) and perhaps some interaction diagrams ([Figure 15.2](#)).

Figure 15.1. A collaboration with its class diagram of roles

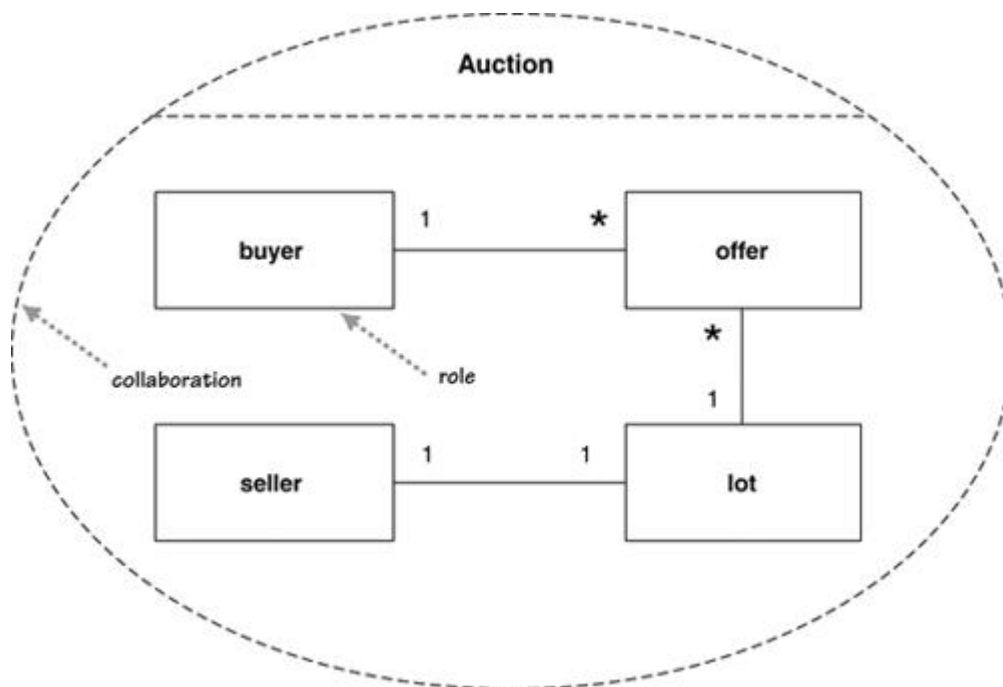
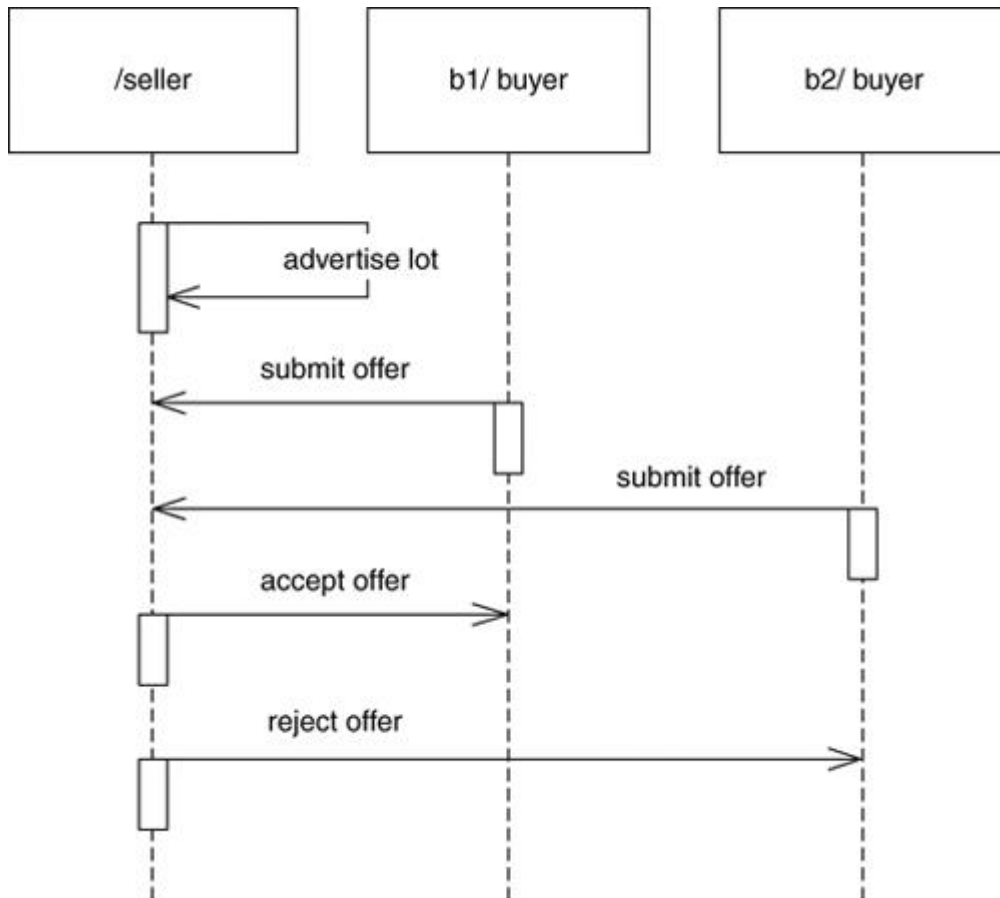


Figure 15.2. A sequence diagram for the auction collaboration

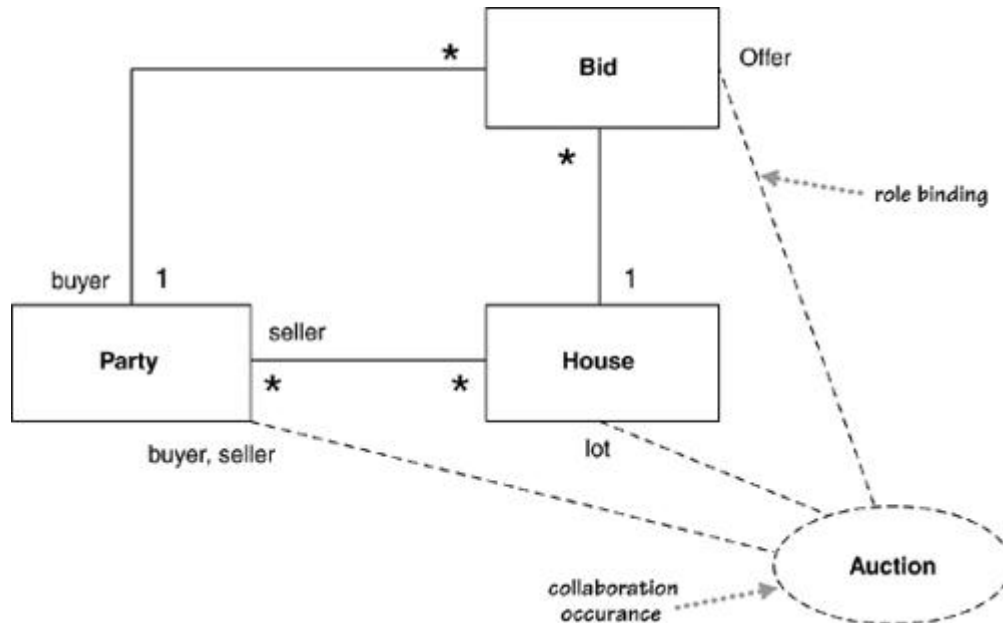


[Figure 15.1](#) is not quite a regular class diagram. For a start, it's surrounded by the dashed ellipse, which represents the auction collaboration. Second, the so-called classes in the collaboration are not classes but roles that will be realized as the collaboration is applied—hence the fact that their names aren't capitalized. It's not unusual to see actual interfaces or classes that correspond to the collaboration roles, but you don't have to have them.

In the interaction diagram, the participants are labeled slightly differently from the usual case. In a collaboration, the naming scheme is `participant-name /role-name : class-name`. As usual, all these elements are optional.

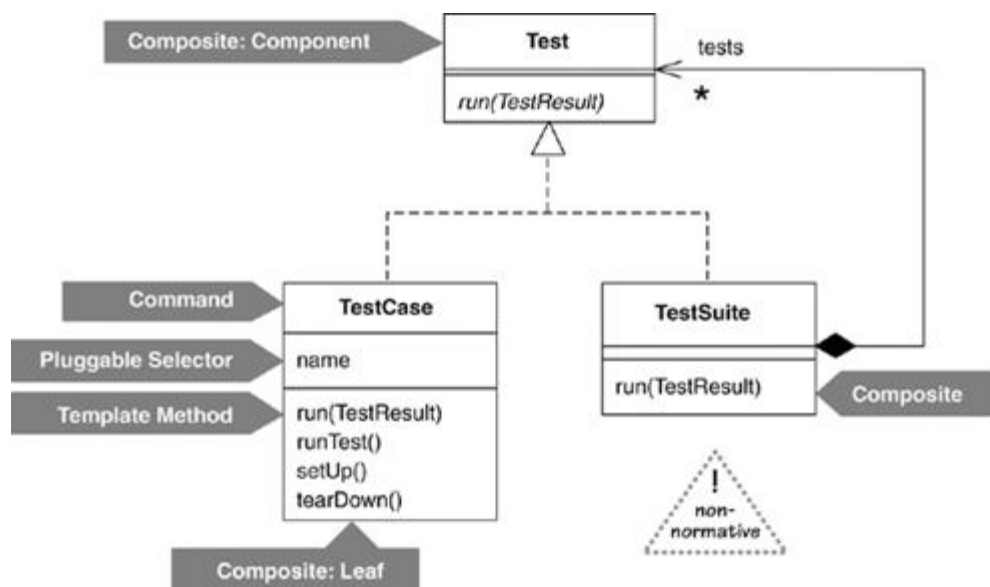
When you use a collaboration, you can show that by placing a collaboration occurrence on a class diagram, as in [Figure 15.3](#), a class diagram of some of the classes in the application. The links from the collaboration to those classes indicate how the classes play the various roles defined in the collaboration.

Figure 15.3. A collaboration occurrence



The UML suggests that you can use the collaboration occurrence notation to show the use of patterns, but hardly any patterns author has done this. Erich Gamma developed a nice alternative notation ([Figure 15.4](#)). Elements of the diagram are labeled with either the pattern name or a combination of *pattern:role*.

Figure 15.4. A nonstandard way of showing pattern use in JUnit (junit.org)



When to Use Collaborations

Collaborations have been around since UML 1, but I admit I've hardly used them, even in my patterns writing. Collaborations do provide a way to group chunks of interaction behavior when roles are played by different classes. In practice, however, I've not found that they've been a compelling diagram type.

Chapter 16. Interaction Overview Diagrams

Interaction overview diagrams are a grafting together of activity diagrams and sequence diagrams. You can think of interaction overview diagrams either as activity diagrams in which the activities are

Changes from UML 1.0 to 1.1

Type and Implementation Class

In the first edition of *UML Distilled*, I talked about perspectives and how they altered the way people draw and interpret models—in particular, class diagrams. UML now takes this into account by saying that all classes on a class diagram can be specialized as either types or implementation classes.

An implementation class corresponds to a class in the software environment in which you are developing. A type is rather more nebulous; it represents a less implementation-bound abstraction. This could be a CORBA type, a specification perspective of a class, or a conceptual perspective. If necessary, you can add stereotypes to differentiate further.

You can state that for a particular diagram, all classes follow a particular stereotype. This is what you would do when drawing a diagram from a particular perspective. The implementation perspective would use implementation classes, whereas the specification and conceptual perspective would use types.

You use the realization relationship to indicate that an implementation class implements one or more types.

There is a distinction between type and interface. An interface is intended to directly correspond to a Java or COM-style interface. Interfaces thus have only operations and no attributes.

You may use only single, static classification with implementation classes, but you can use multiple and dynamic classification with types. (I assume that this is because the major OO languages follow single, static classification. If one fine day you use a language that supports multiple or dynamic classification, that restriction really should not apply.)

Complete and Incomplete Discriminator Constraints

In previous printings of *UML Distilled*, I said that the `{complete}` constraint on a generalization indicated that all instances of the supertype must also be an instance of a subtype within that partition. UML 1.1 defines instead that `{complete}` indicates that all subtypes within that partition have been specified, which is not quite the same thing. I have found some inconsistency on the interpretation of this constraint, so you should be wary of it. If you do want to indicate that all instances of the supertype should be an instance of one of the subtypes, I suggest using another constraint to avoid confusion. Currently, I am using `{mandatory}`.

Composition

In UML 1.0, using composition implied that the link was immutable, or frozen, at least for single-valued components. That constraint is no longer part of the definition.

Immutability and Frozen

UML defines the constraint `{frozen}` to define immutability on association roles. As it's currently defined, it doesn't seem to apply it to attributes or classes. In my practice, I now use the term frozen instead of immutability, and I'm happy to apply the constraint to association roles, classes, and attributes.

Returns on Sequence Diagrams

In UML 1.0, a return on a sequence diagram was distinguished by using a stick arrowhead instead of a solid arrowhead (see previous printings). This was something of a pain, as the distinction was too subtle and easy to miss. UML 1.1 uses a dashed arrow for a return, which pleases me, as it makes returns much more obvious. (As I used dashed returns in *Analysis Patterns* [Fowler, AP], it also

The synchronization bar is now referred to as a fork—when splitting control—or as a join—when synchronizing control together. However, you can no longer add arbitrary conditions to joins. Also, you must follow matching rules to ensure that forks and joins match up. Essentially, this means that each fork must have a corresponding join that joins the threads started by that fork. You can nest fork and joins, though, and you can eliminate forks and joins on the diagram when threads go directly from one fork to another fork, or one join to another join.

Joins are fired only when all incoming threads complete. However, you can have a condition on a thread coming out of a fork. If that condition is false, that thread is considered complete for joining purposes.

The multiple-trigger feature is no longer present. In its place, you can have dynamic concurrency in an activity, shown with a * inside an activity box. Such an activity may be invoked several times in parallel; all its invocations must complete before any outgoing transition can be taken. This is loosely equivalent to, although less flexible than, a multiple trigger and matching synchronization condition.

These rules reduce some of flexibility of activity diagrams but do ensure that activity diagrams are truly special cases of state machines. The relationship between activity diagrams and state machines was a matter of some debate in the RTF. Future versions of the UML (after 1.4) may well make activity diagrams a completely different form of diagram.

Changes from UML 1.3 to 1.4

The most visible change in UML 1.4 is the addition of profiles, which allows a group of extensions to be collected together into a coherent set. The UML documentation includes a couple of example profiles. Together with this, there's greater formalism involved in defining a stereotype, and model elements can now have multiple stereotypes; they were limited to one stereotype in UML 1.3.

Artifacts were added to the UML. An artifact is a physical manifestation of a component, so, for example, Xerces is a component and all those copies of the Xerces jar on my disk drive are artifacts that implement the Xerces component.

Prior to 1.3, there was nothing in the UML meta-model to handle Java's package visibility. Now there is, and the symbol is "~".

UML 1.4 also made the stick arrowhead in interaction diagrams mark asynchronous, a rather awkward backward-incompatible change. That caught out a few people, including me.

Changes from UML 1.4. to 1.5

The principal change here was adding action semantics to the UML, a necessary step to make UML a programming language. This was done to allow people to work on this without waiting for the full UML 2.

From UML 1.x to UML 2.0

UML 2 represents the biggest change that's happened yet to the UML. All sorts of things have changed with this revision, and many changes have affected *UML Distilled*.

Within the UML, there have been deep changes to the UML meta-model. Although these changes don't affect the discussion in *UML Distilled*, they are very important to some groups.

One of the most obvious changes is the introduction of new diagram types. Object diagrams and package diagrams were widely drawn before but weren't official diagram types; now they are. UML 2 changed the name of collaboration diagrams to communication diagrams. UML 2 has also introduced new diagram types: interaction overview diagrams, timing diagrams, and composite structure diagrams.

A lot of changes haven't touched *UML Distilled*. I've left out discussion of such constructs as state machine extensions, gates in interactions diagrams, and power types in class diagrams.

So for this section, I'm discussing only changes that make it into *UML Distilled*. These are either changes to things I discussed in previous editions or new things I've started to discuss with this edition. Because the changes are so widespread, I've organized them according to the chapters in this book.

Class Diagrams: The Essentials ([Chapter 3](#))

Attributes and unidirectional associations are now primarily simply different notations for the same underlying concept of property. Discontinuous multiplicities, such as [2, 4], have been dropped. The frozen property has been dropped. I've added a list of common dependency keywords, several of which are new to UML 2. The `«parameter»`, and `«local»` keywords have been dropped.

Sequence Diagrams ([Chapter 4](#))

The big change here is the interaction frame notation for sequence diagrams to handle iterative, conditional, and various other controls of behavior. This now allows you to express algorithms pretty completely in sequence diagrams, although I'm not convinced that these are any clearer than code. The old iteration markers and guards on messages have been dropped from sequence diagrams. The heads of the lifelines are no longer instances; I use the term participant to refer to them. The collaboration diagrams of UML 1 were renamed to communication diagrams for UML 2.

Class Diagrams: Concepts ([Chapter 5](#))

Stereotypes are now more tightly defined. As a result, I now refer to words in guillemets as keywords, only some of which are stereotypes. Instances on object diagrams are now instance specifications. Classes can now require interfaces as well as provide them. Multiple classification uses generalization sets to group generalizations into groups. Components are no longer drawn with their special symbol. Active objects have double vertical lines instead of thick lines.

State Machine Diagrams ([Chapter 10](#))

UML 1 separated short-lived actions from long-lived activities. UML 2 calls both activities and uses the term do-activity for the long-lived activities.

Activity Diagrams ([Chapter 11](#))

UML 1 treated activity diagrams as a special case of state diagram. UML 2 broke that link and as a result removed the rules of matching forks and joins that UML 2 activity diagrams had to keep to. As a result, they are best understood by token flow rather than by state transition. A whole bunch of new notation thus appeared, including time and accept signals, parameters, join specifications, pins, flow transformations, subdiagram rakes, expansion regions, and flow finals.

A simple but awkward change is that UML 1 treated multiple incoming flows to an activity as an implicit merge, while UML 2 treats them as an implicit join. For this reason, I advise using an explicit merge or join when doing activity diagrams.

Swim lanes can now be multidimensional and are generally called partitions.